Rocket Code GmbH

Rocket Code GmbH
Am Deich 86
28199 Bremen
www.rocket-code.de

Rocket Code GmbH · Am Deich 86 · 28199 Bremen

Khaled Swaidan
Graf-Molte-Straße. 57
28211 Bremen

Ihr Ansprechpartner:
Michel Otto
+49 (421) 95 800 693
m.otto@rocket-code.de
info@rocket-code.de

08.11.2026

Arbeitszeugnis

Herr Khaled Swaidan, geboren am 06.07.1996 war vom 15.02.2025 bis zum 30.04.2026 studienbegleitend als Werkstudent im Bereich Softwareentwicklung (Frontend) in unserem Unternehmen tätig.

Die 2024 gegründete Rocket Code GmbH mit Sitz in Bremen entwickelt, implementiert und betreut individuelle Softwarelösungen. Darüber hinaus bietet das Unternehmen IT-Beratung sowie Schulungen und Workshops im Bereich Softwareentwicklung an. Zum Technologie-Stack gehören unter anderem Flutter, C# und Microsoft Azure.

Zu seinen Aufgaben gehörten insbesondere:

- UI-Design: Konzeption und visuelle Gestaltung von Benutzeroberflächen
- Frontend-Entwicklung: programmtechnische Umsetzung der Entwürfe mit Flutter
- Durchführung von Softwaretests
- Anwendersupport für unsere Anwendung

Herr Swaidan verfügt über ein umfassendes Fachwissen in der Frontend-Entwicklung mit Flutter sowie in der Gestaltung von Benutzeroberflächen. Diese setzte er sicher und zielgerichtet in der Praxis ein. Seine Kenntnisse im Bereich der Softwarearchitektur baute er während seiner Tätigkeit stetig aus.

Herr Swaidan zeigte stets großes Engagement und brachte mit viel Eigeninitiative eigene Ideen in seine Arbeit ein. Dank seiner schnellen Auffassungsgabe arbeitete er

sich zügig in neue Aufgabenstellungen ein. Auch bei erhöhtem Arbeitsaufkommen behielt er den Überblick und erzielte gute Ergebnisse.

Er arbeitete sorgfältig und ging neuen Themen mit großer Neugier nach. Bei klar gesetzten Prioritäten setzte er seine Aufgaben zielgerichtet um. Seine Arbeitsergebnisse entsprachen unseren Anforderungen. Besonders die von ihm entworfenen und umgesetzten Benutzeroberflächen überzeugten durch hohe gestalterische Qualität und gute Bedienbarkeit.

Herr Swaidan erledigte die ihm übertragenen Aufgaben stets zu unserer vollen Zufriedenheit.

Sein Verhalten gegenüber Vorgesetzten, Kollegen und Anwendern war stets vorbildlich. Mit seiner freundlichen, hilfsbereiten und offenen Art war er im gesamten Team sehr geschätzt und trug wesentlich zu einem ausgezeichneten Arbeitsklima bei.

Das Arbeitsverhältnis endet mit Ablauf des 30.04.2026. Wir bedauern sein Ausscheiden sehr und danken ihm für seine wertvollen Beiträge und die stets angenehme Zusammenarbeit. Für seinen weiteren beruflichen und persönlichen Lebensweg wünschen wir ihm alles Gute und weiterhin viel Erfolg.

Bremen, 08.11.2026

Michel Otto
Geschäftsführer
Rocket Code GmbH



Universität Bremen

Faculty 3 — Mathematics and Computer Science

Master's Thesis

A Retrieval-Augmented AI Assistant for Supporting Model Construction and User Inquiries in SIRcraft

Khaled Swaidan

Matriculation No. 3144723

September 23, 2026

First Supervisor: Prof. Dr. Peter Haddawy
Second Supervisor: Dr. Thomas Barkowsky

Abstract

SIRcraft is a free, offline, no-code desktop application for building deterministic compartmental models of infectious diseases. It removes the software barrier to epidemiological modelling but not the conceptual one: users still need to translate biological mechanisms into valid rate equations, and general-purpose language models cannot assist reliably because they are not grounded in SIRcraft's documentation, default to LaTeX markup that SIRcraft's parser rejects, and cannot distinguish supported features from plausible-sounding ones that do not exist.

This thesis designs, implements, and evaluates a domain-grounded retrieval-augmented generation (RAG) assistant that runs entirely within the University of Bremen network. The system combines a hand-curated corpus of 179 SIRcraft-related chunks, a local embedding and retrieval pipeline, locally hosted open-weights language models, a canonical system prompt shared across models, and a post-processing gateway that validates generated equations against SIRcraft's equation grammar.

The assistant is evaluated against a 30-question benchmark spanning five functional categories, using two rubrics that score supported questions for factual grounding, completeness, and executability, and unsupported questions for refusal adherence, scope justification, and constructive alternatives. Four locally hosted models are compared under the same prompt and retrieval pipeline.

Grounding holds: no model fabricated a GUI feature on a supported question, and every model refused at least nine of the ten unsupported questions. The strongest model reached 166 of 180 rubric points. The evaluation also identifies concrete limits in the knowledge base, in refusal quality, and in the post-processing stage. The system, corpus, and benchmark are released with the thesis.

Contents

Abstract **2**

Contents **i**

List of Figures **v**

List of Tables **vi**

List of Listings **viii**

1 Introduction **1**

1.1 Background and Motivation 1

1.2 Problem Statement 2

1.3 Research Questions and Objectives 4

1.3.1 Main Research Question and Overarching Objectives 4

1.3.2 Specific Research Sub-Questions 4

1.3.3 Methodological Objectives and Deliverables 4

2 Background and Related Work **6**

2.1 Existing Compartmental Modelling Software 6

2.1.1 Categorization and Taxonomy of Epidemiological Software 6

2.1.2 Applications with Fixed Disease Structures 6

2.1.3 Interactive Code-Based Packages and Script Wrappers 7

2.1.4 Web-Based Graphical Simulators and Cloud Dependencies 7

2.1.5 Comparative Trade-Offs and Motivation for SIRcraft 8

2.2 Retrieval-Augmented Generation (RAG) and Non-Parametric Memory 8

2.2.1 Parametric Memory Limits vs. Non-Parametric Augmentation 8

2.2.2 Dense Vector Spaces and Sentence Embeddings 9

2.2.3 Vector Databases, Semantic Chunking, and Ingestion Pipelines 9

2.3 Mathematical Language Constraints in Domain-Specific Assistants 10

2.3.1 The Natural-to-Mathematical Language Gap in Domain Software 10

2.3.2 Unconstrained Conversational LLMs vs. Rigid Plain-Text Grammars . . . 10

2.3.3 AST Validation as a Post-Processing Gateway Safeguard 11

2.4 Negative Boundary Precision and Local Model Benchmarking . . . 12
2.4.1 Negative Boundary Precision and Hallucination Resistance . . . 12
2.4.2 Local Open-Weights Architectures and Benchmarking . . . 12

3 Concept **14**
3.1 Design Overview . . . 14
3.2 Knowledge Base and Retrieval Design . . . 15
3.2.1 Source Curation . . . 15
3.2.2 Five-Field Chunk Schema . . . 16
3.2.3 Normalization and Semantic Chunking . . . 17
3.2.4 Embedding Model Choice . . . 18
3.2.5 Vector Store and Retrieval Strategy . . . 18
3.3 Generation Design . . . 19
3.3.1 Prompt Architecture . . . 19
3.3.2 System Rules . . . 21
3.3.3 Structured Output Contract . . . 23
3.3.4 Model Orchestration . . . 25
3.4 Post-Processing and AST Validation . . . 26
3.4.1 JSON Recovery: The Salvager . . . 26
3.4.2 Equation Extraction and Normalization . . . 28
3.4.3 Grammar Validation . . . 29
3.5 Frontend and Deployment . . . 33
3.5.1 Chat Interface Architecture . . . 33
3.5.2 Conversation Presentation . . . 34
3.5.3 Deployment and Data Sovereignty . . . 35

4 Implementation **36**
4.1 Development Environment . . . 36
4.2 Backend and Knowledge Base . . . 40
4.2.1 Configuration and Endpoints . . . 40
4.2.2 Ingestion and Indexing . . . 41
4.3 Retrieval and Generation . . . 41
4.3.1 Embedding and Retrieval . . . 42
4.3.2 Generation and Recovery . . . 42
4.3.3 Equation Validation . . . 44
4.4 Frontend (User Interface) . . . 45
4.5 Deployment and Tests . . . 49
4.5.1 Containerisation . . . 49
4.5.2 Nginx Configuration . . . 50

4.5.3 Test Suite . . . 51

5 Evaluation **53**

5.1 Evaluation Design . . . 53

5.1.1 Models and Benchmark Dataset . . . 53

5.1.2 Evaluation Protocol . . . 54

5.1.3 Research Question to Metric Mapping . . . 55

5.2 Scoring Rubrics and Metrics . . . 55

5.2.1 Rubric A: Supported Questions . . . 55

5.2.2 Rubric B: Unsupported Questions . . . 56

5.2.3 Execution-Based Metrics . . . 57

5.3 Results . . . 58

5.3.1 Overall Scores . . . 58

5.3.2 Rubric Dimension Summary . . . 59

5.3.3 Boundary Adherence . . . 60

5.3.4 Syntax Validation Rate . . . 62

5.3.5 Retrieval Observations . . . 63

5.4 Discussion . . . 65

5.4.1 RQ1: Knowledge Base Engineering and Vector Retrieval . . . 65

5.4.2 RQ2: RAG Grounding and Operational Guidance . . . 66

5.4.3 RQ3: Post-Retrieval AST Validation . . . 67

5.4.4 RQ4: Negative Boundary Precision . . . 68

6 Conclusion and Future Work **70**

6.1 Conclusion . . . 70

6.1.1 Problem Restatement . . . 70

6.1.2 Main Research Question . . . 71

6.1.3 Sub-Question Answers . . . 71

6.1.4 Contributions . . . 73

6.2 Future Work . . . 74

Appendices **77**

A Frontend Dependencies **77**

B The 30-Question Benchmark Dataset **78**

C Canonical System Prompt **81**

Declaration of Authorship **84**

D Use of AI-based applications **87**

Glossary **90**

Bibliography **92**

List of Figures

3.1 Deployment topology and architecture of the SIRcraft RAG Assistant 15
3.2 The post-processing pipeline. Each stage feeds the next; failures at the validation stage are surfaced to the user as guardrail warnings rather than silently dropped. 26
3.3 The escalating recovery ladder. Each rung is attempted only if the previous one fails. Solid arrows mark success; dashed arrows mark escalation to the next rung. 27
3.4 Client architecture 33

4.1 Landing view 45
4.2 Waiting state 46
4.3 Completed reply 47
4.4 Expanded panels 48
4.5 Sidebar controls 49

List of Tables

3.1 Behavioural rules in the system message, grouped by purpose. 22
3.2 Effect of the normalization pipeline on representative inputs. The last row is filtered out before normalization because it matches the placeholder template from the system prompt. 30
3.3 Validator behaviour on representative inputs. 31

4.1 Direct Python dependencies for the backend, taken from `requirements.txt`. The file covers runtime and test packages; transitive dependencies are not listed. . . . 37

5.1 Composition of the 30-question benchmark dataset by category. The full list is given in Appendix B. 54
5.2 Mapping from each research question to the metric or exhibit that addresses it. . 55
5.3 Rubric A for supported questions. Each of the three dimensions is scored 0–2, giving a maximum of 6 points per question. 56
5.4 Rubric B for unsupported questions. Each of the three dimensions is scored 0–2, giving a maximum of 6 points per question. 57
5.5 Overall benchmark scores per model. Total is out of 180 (Rubric A out of 120 across the 20 supported questions, Rubric B out of 60 across the 10 unsupported questions). Each category column is structured as *total* (*A* | *B*), where the ceiling is 36 (24 | 12) — 36 category points split into 24 Rubric A points (4 supported questions at 6 each) and 12 Rubric B points (2 unsupported questions at 6 each). For example, `deepseek-r1:32b` on Category 1 scored 32 (23 | 9): 32 of the 36 available, split as 23 A-points and 9 B-points. 58
5.6 Rubric dimension summary per model. Each cell is *earned* / *available* (*percentage*) summed across all questions to which the dimension applies. The three summary rows aggregate the S-dimensions into Rubric A, the U-dimensions into Rubric B, and both into the overall total. 60

5.7 Boundary adherence per model, decomposed by Rubric B dimension. Each of the top three rows scores 0–2 per unsupported question, summed across the ten questions for a maximum of 20. The final row reports the total Rubric B score. . 61
5.8 AST Syntax Validation Pass Rate per model, computed on the raw equations from each model's `equations` array. Columns report the model's pass rate, the invalid count over the total, and the split of those invalid entries between ANTLR syntax errors and function-registry errors. 62

Listings

3.1 JSON payload structure 20
4.1 Repository layout 39
4.2 The Salvager regex 43
C.1 Canonical system prompt, part 1 82
C.2 Canonical system prompt, part 2 83

Chapter 1

Introduction

1.1 Background and Motivation

Dengue fever and other vector-borne diseases have become among the most pressing public health challenges of the present decade [Gre25, Wor25]. On the American continent, more than 12.4 million suspected dengue cases were reported between epidemiological weeks 1 and 44 of 2024—a 381% increase over the average of the preceding five years [Pan24]. The World Health Organization (WHO) currently places the annual global burden at 100–400 million infections, of which roughly 96 million are clinically apparent [BGB+13, Wor25]. A warming climate and the changing distribution of *Aedes* mosquitoes are among several factors linked to dengue's emergence in countries where it was previously absent [Wor25].

To anticipate these outbreak trajectories, evaluate health risks, and design targeted public health interventions—such as vaccination campaigns, vector control via insecticide spraying, or movement lockdowns—epidemiologists and researchers rely heavily on mathematical compartmental models [Gre25]. First formalized in the SIR model by Kermack and McKendrick [KM27], compartmental models divide a population into mutually exclusive health states, such as Susceptible (S), Infected (I), and Recovered (R), or with additional vector compartments (S_v, I_v) for vector-borne diseases, and track population flow over time using systems of coupled, first-order Ordinary Differential Equations (ODEs) [Gre25]. In host-vector transmission frameworks such as SIR-SI and SIRS-SI, the spread of infection between human hosts and mosquito vectors is governed by biting rate equations and cross-species infection probabilities [SBZ+21, Gre25].

Despite their widespread use, compartmental simulation software has long been constrained by technical and structural barriers that limit its practical adoption in research, education, and public policy [Gre25]:

- **Programming and Mathematical Complexity:** Code-based modeling libraries such as `shinySIR` in R, `ModSimPy` in Python, or `SimBiology` in MATLAB offer high modeling flexibility but require familiarity with a programming language and with differential equa-

tion syntax, which presents a steep barrier for public health practitioners without formal computer science training.

- **Commercial License Costs:** Feature-rich system dynamics applications such as Berkeley Madonna, Stella, AnyLogic, and Vensim provide visual model editors but require paid commercial licenses, restricting access in resource-constrained public health institutions and universities.
- **Fixed, Predefined Model Scenarios:** Non-commercial simulation tools such as InfluSim, FluSurge, CommunityFlu, and FluTE restrict users to fixed, predefined disease scenarios—influenza outbreaks—and do not allow novel compartmental structures to be constructed.
- **External Web Dependencies:** Online graphical modeling platforms such as InsightMaker provide customizable web editors but require internet connectivity and user account registration, and their continued availability relies on remote infrastructure maintained by third parties.

Grebe's platform addresses all four software barriers: it is free and offline, and its no-code graphical editor allows arbitrary compartmental models to be built without programming. A different kind of barrier remains, however: users without formal epidemiological training still need to translate biological mechanisms into valid rate equations.

This Master's thesis therefore designs, implements, and empirically evaluates a domain-grounded Retrieval-Augmented Generation (RAG) AI assistant built on Grebe's desktop platform [Gre25]. The system is designed to help users navigate SIRcraft, overcome conceptual modelling barriers, recognise and decline requests outside SIRcraft's supported scope, such as stochastic or spatial modelling, and ensure that generated mathematical expressions pass SIRcraft's internal Abstract Syntax Tree (AST) parser without syntax errors.

1.2 Problem Statement

SIRcraft removes the programming and environment-setup barriers of code-based tools through its no-code graphical interface [Gre25]. However, users still face a steep conceptual learning curve: translating biological transmission mechanisms into valid rate equations, configuring time-series parameters, setting up piecewise interventions, and navigating multi-step GUI workflows all require domain knowledge that the platform itself does not teach [HSH+25]. Public health researchers, epidemiologists, and students increasingly turn to general-purpose Large Language Models (LLMs) such as ChatGPT or Claude for assistance with these tasks.

Off-the-shelf LLMs are not grounded in SIRcraft's documentation and may produce plausible but fictitious guidance, such as suggesting UI controls that do not exist or recommending modelling paradigms that SIRcraft does not support—for example, individual agent-based modelling, spatio-temporal GIS mapping, or stochastic differential equations with Brownian noise terms

($dW(t)$) [HSH+25, LPP+20, Gre25]. Such recommendations contradict SIRcraft's explicit scope, which is restricted to deterministic, well-mixed Ordinary Differential Equation (ODE) compartmental models [Gre25]. Assisting SIRcraft users effectively therefore requires a domain-grounded retrieval system that enforces negative system boundaries, preventing ungrounded language models from offering misleading advice or suggesting invalid workflows. In addition to system boundaries, handling mathematical expressions presents a critical technical hurdle. To make equation entry accessible without requiring complex markup languages, SIRcraft uses a custom, human-readable mathematical grammar built with the ANTLR parser generator [Gre25]. This grammar parses user-entered formulas into a Concrete Syntax Tree (CST) and transforms them into an Abstract Syntax Tree (AST) for numerical integration via SciPy's LSODA solver [Gre25]. In the development of the assistant, however, it became apparent that unconstrained LLMs tend to format mathematical outputs using academic LaTeX markup (e.g., `\frac{\beta S I}{N}`), multiline code blocks, or thousands separators in numerical values (e.g., `1,000,000`). When LaTeX constructs are pasted into SIRcraft's transition rate or variable property fields, the parser rejects them outright and the expression cannot be used without manual correction. Thousands separators behave differently: the equation field accepts the comma syntactically, but the underlying evaluator stops at the comma and reads only the leading digit sequence. The expression `2,000 * E` is therefore accepted silently and evaluated as `2`, producing a model that compiles and runs without warning but computes the wrong value. The failure is silent rather than loud, which makes it harder to detect from the UI alone.

Finally, the deployment context introduces strict infrastructure constraints. This AI assistant is developed within the University of Bremen, which operates under institutional data-governance and privacy requirements. Transmitting preliminary epidemiological models, unreleased outbreak datasets, or institutional research materials to external cloud LLM API providers would raise data-sovereignty and service-dependency concerns for this setting. To address these requirements while supporting SIRcraft users, the RAG assistant is deployed as a web-based client-server architecture hosted entirely on University of Bremen infrastructure. Users access the system via a browser-based Flutter web interface hosted on a central university web server. The web server runs the FastAPI backend gateway and the ChromaDB vector store, and communicates over the secure university network with a dedicated on-premise AI server running Ollama, which serves the local embedding model and orchestrates open-weights models from the Gemma and DeepSeek-R1 families. By keeping all vector storage, application gateways, and LLM inference within the University of Bremen network, the system achieves zero external API dependencies and supports the institution's privacy requirements.

1.3 Research Questions and Objectives

1.3.1 Main Research Question and Overarching Objectives

The main research question driving this work is:

> **Main Research Question (MRQ):** *How effectively can a local, domain-grounded Retrieval-Augmented Generation (RAG) system assist users in constructing epidemiological compartmental models, guiding GUI operations, and enforcing negative system boundaries—that is, recognising and declining requests that fall outside SIRcraft's supported scope—?*

1.3.2 Specific Research Sub-Questions

To answer the main research question, the inquiry is divided into four sub-questions that address the core failure modes of general-purpose LLMs identified in Section 1.2:

RQ1 — Knowledge Base Engineering & Vector Retrieval:
How can SIRcraft software manuals, video tutorial transcripts, thesis documentation, and built-in example specifications be normalized and semantically chunked into a dense vector index (ChromaDB) to enable precise, domain-grounded retrieval?

RQ2 — RAG Grounding & Operational Guidance:
How do different local open-weights language models, when coupled with a non-parametric dense vector store [LPP+20], differ in their ability to provide factual, actionable software guidance while avoiding fabricated GUI features and workflows?

RQ3 — Post-Retrieval AST Validation & Syntactic Compliance:
To what extent can a specialized post-processing gateway extract candidate mathematical expressions and enforce compliance with the equation syntax accepted by SIRcraft's custom ANTLR grammar [Gre25] via independent AST validation, without manual user correction?

RQ4 — Negative Boundary Precision & Refusal Awareness:
How reliably do local RAG architectures recognise and refuse unsupported feature requests—such as stochastic differential equations, spatio-temporal GIS mapping, or agent-based modelling—while providing valid, constructive workarounds [Gre25]?

1.3.3 Methodological Objectives and Deliverables

In support of these research questions, the thesis pursues three engineering and evaluation deliverables:

1. **Knowledge Base Engineering:** Compile, normalize, and semantically index four categories of SIRcraft source material—thesis documentation, built-in example specifications,

video tutorial transcripts, and application manual entries—into a ChromaDB vector store using local text embeddings.

2. **Full-Stack System Implementation & Dual-Server Deployment:** Architect and deploy the assistant as a complete web-based client-server platform on two University of Bremen servers, as described in Section 1.2. The platform combines a Flutter web application, a FastAPI backend gateway, a ChromaDB vector database, an independent AST validation pipeline designed to accept the same surface syntax as SIRcraft's equation grammar [Gre25], and a dedicated local AI server running Ollama for open-weights LLM inference.
3. **Empirical Benchmark Evaluation:** Construct a 30-question benchmark dataset spanning five functional categories and evaluate four local LLMs (`gemma2:9b`, `gemma4:26b`, `gemma2:27b`, and `deepseek-r1:32b`) on an Nvidia RTX 5090 workstation. The five categories are:
   (a) Platform setup, packaging, and UI navigation;
   (b) Compartmental canvas architecture and multi-species modelling;
   (c) Mathematical grammar, AST syntax, and solvers;
   (d) Dynamic scenarios, visualizations, and validation;
   (e) Built-in benchmark case studies.

Chapter 2

Background and Related Work

2.1 Existing Compartmental Modelling Software

2.1.1 Categorization and Taxonomy of Epidemiological Software

Existing epidemiological simulation software falls into three broad categories, each representing a different compromise between modeling flexibility, accessibility, and deployment cost:

1. **Applications with fixed disease structures**, including deterministic compartmental models such as InfluSim [ESDB07] and stochastic agent-based simulators such as FluTE [CHOL10];
2. **Code-based packages** that require a programming environment, such as `shinySIR` [Mor20];
3. **General-purpose web graphical tools**, such as Insight Maker [FR14].

The software barriers identified in Chapter 1 manifest differently across these three categories. The following subsections examine each category and the barriers it presents.

2.1.2 Applications with Fixed Disease Structures

A major class of public health planning tools consists of applications built around fixed, predefined disease structures [Gre25].

InfluSim [ESDB07] is a representative example. Developed by Eichner et al. for influenza pandemic preparedness planning, it is a deterministic compartmental model extending the classical SEIR framework with over 1,000 differential equations. The user interface allows rapid parameter adjustment through sliders and input fields, but the underlying model topology is fixed: researchers cannot add compartments or modify the differential equation structure.

FluTE [CHOL10], developed by Chao et al., takes a stochastic, individual-based approach. It simulates influenza transmission across synthetic social contact networks spanning metropolitan

areas to the continental United States, with parameters supplied through text-based configuration files. Unlike InfluSim, FluTE provides no built-in graphical interface or visualization; results are written to text files for external parsing, imposing additional usability overhead on users without programming experience.

2.1.3 Interactive Code-Based Packages and Script Wrappers

To overcome the rigid constraints of fixed-scenario tools, developers have created code-based libraries and interactive plotting tools within programming environments [Gre25]. A representative example is `shinySIR` [Mor20], an open-source R package developed by Morris that provides interactive web-based plotting for ordinary differential equation (ODE) compartmental models via the R Shiny framework. `shinySIR` includes built-in standard models such as SIR, SIRS, and SIS, and allows users to define custom ODE equations in R syntax. Designed as a teaching tool, it enables students to visualize parameter impacts with minimal coding knowledge.

Nevertheless, code-based packages still require users to maintain local programming environments, install package dependencies, and write valid equation functions in script files. This dependency presents a steep barrier for domain experts, epidemiologists, and students without formal computer science training [Gre25].

2.1.4 Web-Based Graphical Simulators and Cloud Dependencies

Among customizable graphical modeling tools, web-based System Dynamics platforms are the closest architectural relatives of SIRcraft [Gre25]. A widely used example is Insight Maker [FR14], a general-purpose web-based simulation environment that unifies System Dynamics stock-and-flow diagramming, agent-based modeling, and imperative scripting within a single canvas. Users build models from visual primitives such as stocks, flows, variables, and links, inspect interactive charts, and publish models to a centralized repository [FR14].

For specialized epidemiological workflows, however, several functional gaps become apparent [Gre25]. Insight Maker does not natively accept empirical time-series data as driving model inputs, cannot export model equations in a publication-ready format, and does not persist custom multi-chart dashboard layouts alongside saved models. Its reliance on an external cloud server also introduces practical constraints: internet connectivity and user account registration are required, and the tool's availability depends on the continued operation of the remote infrastructure [Gre25].

2.1.5 Comparative Trade-Offs and Motivation for SIRcraft

Each category of epidemiological modeling tools presents a different compromise between flexibility, accessibility, and deployment cost. Fixed-scenario applications such as InfluSim [ESDB07] and FluTE [CHOL10] minimize setup complexity but cannot accommodate arbitrary model topologies. Code-based libraries offer extensive customization but require programming literacy and local environment maintenance. Commercial system dynamics suites provide feature-rich visual editors but impose licensing fees that restrict access in resource-constrained institutions. Free web platforms such as Insight Maker [FR14] eliminate licensing costs but depend on remote infrastructure and lack domain-specific capabilities such as time-series driving variables, publication-ready equation export, and saved multi-chart dashboards [Gre25].

To address these barriers, SIRcraft was developed as a free, license-free, standalone desktop application that provides a visual, no-code canvas for arbitrary deterministic ODE models while remaining independent of external web services [Gre25].

The software landscape reviewed above provides the context for SIRcraft and, through it, for the assistant this thesis develops. The following sections turn to the AI technologies that underlie the assistant: retrieval-augmented generation, syntactic validation of generated equations, and enforcement of negative system boundaries.

2.2 Retrieval-Augmented Generation (RAG) and Non-Parametric Memory

2.2.1 Parametric Memory Limits vs. Non-Parametric Augmentation

Large pre-trained language models (LLMs) encode substantial world knowledge within their neural network parameters, functioning as implicit parametric knowledge bases [LPP+20]. This purely parametric approach has three limitations for domain-specific applications: internal knowledge cannot be revised without expensive retraining, the model cannot cite its sources, and it may generate factually incorrect statements—commonly termed hallucinations—when queried on specialized topics [LPP+20]. In mathematical contexts, unconstrained parametric LLMs additionally struggle to bridge the gap between natural language and formal mathematical structures [HSH+25]. When applied to specialized platforms such as SIRcraft, such models may emit LaTeX notation that fails in SIRcraft's ANTLR-based equation parser or falsely claim support for unsupported paradigms such as agent-based or spatio-temporal modeling [Gre25, HSH+25].

To mitigate these limitations, hybrid retrieval-augmented generation (RAG) architectures combine parametric generation with an explicit, non-parametric memory [LPP+20]. This non-parametric memory takes the form of a dense vector index over external domain knowledge—

software manuals, technical documentation, tutorial transcripts—accessed at inference time by a neural retriever. By conditioning generation on retrieved passages, RAG models produce outputs that are more factual, specific, and diverse, while reducing hallucination rates [LPP+20]. A further advantage is that the underlying index can be updated at test time without altering the model parameters.

2.2.2 Dense Vector Spaces and Sentence Embeddings

Efficient non-parametric retrieval requires representing queries and documents as dense vectors in a shared space [LPP+20, RG19]. Standard BERT-based approaches use a cross-encoder architecture, in which query and document are concatenated and processed jointly through the transformer network [RG19]. Cross-encoders achieve strong accuracy but incur substantial cost: identifying the most similar pair among $N = 10{,}000$ sentences requires roughly 50 million inference passes ($O(N^2)$ complexity), making them impractical for interactive search.

To overcome this, bi-encoder architectures such as Sentence-BERT (SBERT) use Siamese and triplet network structures to embed sentences independently into fixed-size dense vectors [RG19]. Because similar sentences project to nearby points, bi-encoders decouple document indexing from query processing, and the search cost for 10,000 sentences drops from approximately 65 hours with a cross-encoder to a few seconds.

The SIRcraft assistant follows this principle using a modern embedding model, `nomic-embed-text` [NMDM25], rather than SBERT itself. Like SBERT, it produces independent dense vector representations for each input text, allowing document embeddings to be computed offline. Because `nomic-embed-text` is trained with task-specific prefixes, the ingestion pipeline embeds each documentation chunk with a `search_document:` prefix, while queries are embedded with a `search_query:` prefix [NMDM25]. This asymmetry aligns queries and documents within a shared semantic space.

Retrieval is handled by ChromaDB [Chr24], which stores the dense vector index and performs approximate nearest neighbor search. ChromaDB uses a Hierarchical Navigable Small World (HNSW) index [MY20], which provides sub-linear query time and enables the gateway to return the top $K = 7$ chunks within milliseconds. Both the embedding model and the language model are served locally through Ollama [Oll24], keeping all inference inside the University of Bremen network.

2.2.3 Vector Databases, Semantic Chunking, and Ingestion Pipelines

Applying dense vector retrieval to a domain-specific assistant requires a systematic ingestion pipeline [LPP+20]. Raw documentation cannot be embedded directly due to context window

limits and token noise: earlier open-source embedding models are limited to 512-token inputs, while modern models such as `nomic-embed-text` support context windows of up to 8,192 tokens [NMDM25]. Source material must therefore be normalized and split into semantically coherent units before embedding. Unlike the fixed-length chunking used by Lewis et al. [LPP+20], the SIRcraft pipeline splits documentation at semantic boundaries derived from the structure of the source material. The specific normalization and chunking procedure is described in Chapter 3.

2.3 Mathematical Language Constraints in Domain-Specific Assistants

Two related challenges arise when language models are asked to produce equations for a domain-specific platform. The first is the general difficulty of translating natural language into formal mathematical language. The second is the specific formatting constraint imposed by the target software's equation parser. The following subsections address each in turn.

2.3.1 The Natural-to-Mathematical Language Gap in Domain Software

Translating informal natural-language descriptions of dynamic phenomena into formal mathematical structures remains a significant challenge for artificial intelligence [HSH+25]. Natural language captures the complexity and nuance of human thought, while mathematical language excels in precision. Empirical evaluations on specialized benchmarks such as Mamo reveal that even state-of-the-art language models struggle to formulate coupled Ordinary Differential Equations (ODEs) from natural-language descriptions. A single statement in natural language may correspond to different mathematical representations, and subtle formatting deviations or missing parameters can cause compilation failures in rigid software environments [HSH+25]. In domain-specific software modeling, it is therefore essential to distinguish conceptual modeling errors from superficial syntax or formatting errors. For the SIRcraft assistant, this distinction is critical: the language model must translate user requests into valid SIRcraft transition equations, and any superficial formatting deviation may cause the SIRcraft parser to reject an otherwise conceptually correct model. The following subsections examine the safeguard that the RAG gateway implements to address this.

2.3.2 Unconstrained Conversational LLMs vs. Rigid Plain-Text Grammars

When interacting with specialized modeling platforms such as SIRcraft, unconstrained language generation may produce formatting failures that are unrelated to the underlying modeling

logic [HSH+25]. During development of the SIRcraft assistant, it was observed that general-purpose LLMs tend to present mathematical notation as academic LaTeX, multi-line code blocks, or reasoning traces such as the `<think>` blocks emitted by models like DeepSeek-R1 [Dee25], and that they may additionally introduce prohibited thousands separators (e.g., `1,000,000` instead of `1000000`) or differential operator notation (e.g., `dS/dt = ...`).

SIRcraft's simulation engine, by contrast, relies on a human-writable plain-text grammar built with the ANTLR 4 parser generator [Gre25, Par13]. This grammar parses transition formulas into a Concrete Syntax Tree (CST) and transforms them into an Abstract Syntax Tree (AST) for numerical integration via SciPy's LSODA solver. Because the grammar accepts only this subset of plain-text syntax, LaTeX tags and unparsed code fences are rejected by the parser, preventing the simulation engine from compiling the system of differential equations. Thousands separators behave differently: the equation field accepts the comma at parse time, but the evaluator stops at the comma and reads only the leading digit sequence, so `2,000 * E` is silently evaluated as `2`. The failure is numerical rather than syntactic, and it produces a model that compiles without warning but computes the wrong value.

In the SIRcraft RAG assistant, this problem is directly relevant. Every time a user submits a question, the language model generates a response that may contain equations, and those equations must be extracted, normalized, and validated before they can be used in SIRcraft. The next subsection describes the safeguard that the RAG gateway implements to enforce this validation.

2.3.3 AST Validation as a Post-Processing Gateway Safeguard

This safeguard addresses the formatting failures described in the previous subsection. Huang et al. [HSH+25] propose a post-generation approach in which code modifiers or syntax validators correct formatting defects without altering the underlying mathematical logic.

Within the SIRcraft assistant, the post-processing stage is implemented inside the RAG service and executes after generation returns. It uses regular-expression heuristics to scrub Chain-of-Thought (`<think>`) reasoning traces, isolate mathematical candidate strings from Markdown responses, and strip residual LaTeX formatting artifacts; one rung of its JSON-recovery ladder is internally termed the Salvager. The cleaned expressions are then passed to a recursive-descent validator designed to accept the same surface syntax as SIRcraft's ANTLR `Expression.g4` grammar [Gre25]. Verifying that candidate formulas parse successfully before user display ensures that generated equations can be pasted directly into SIRcraft's transition property panels without triggering application syntax errors.

2.4 Negative Boundary Precision and Local Model Benchmarking

2.4.1 Negative Boundary Precision and Hallucination Resistance

Building on the operational boundaries outlined in Chapter 1 and the RAG architecture established in Section 2.2, evaluating the assistant's safety requires assessing **negative boundary precision**. While standard RAG benchmarks evaluate positive retrieval accuracy, negative boundary precision measures the system's ability to identify and decline user queries that request features outside SIRcraft's documented capabilities.

The SIRcraft assistant enforces this boundary through its system prompt, which includes explicit refusal examples and instructs the model to base its answers strictly on retrieved documentation rather than on its own parametric knowledge. When a user asks about an unsupported paradigm, such as stochastic differential equations, spatio-temporal GIS mapping, or agent-based models, the model is instructed to decline and redirect the user toward valid deterministic ODE workarounds. This prompt-based safeguard reduces the model's parametric memory from fabricating non-existent software controls and keeps responses within SIRcraft's documented scope.

2.4.2 Local Open-Weights Architectures and Benchmarking

As established in Section 1.2, deploying the assistant on local infrastructure is essential to comply with institutional data-sovereignty requirements. This design choice aligns with SIRcraft's own philosophy of avoiding dependence on external services [Gre25]. A dedicated local inference environment, such as a workstation equipped with an NVIDIA RTX 5090 GPU, keeps all inference inside the institutional network.

The models evaluated in this thesis span three training approaches. The Gemma 2 models (`gemma2:9b` and `gemma2:27b`) are decoder-only transformers optimized via Supervised Fine-Tuning (SFT) and Reinforcement Learning from Human Feedback (RLHF) to map input prompts directly to concise responses [Gem24]. The Gemma 4 model (`gemma4:26b`) is a Mixture-of-Experts architecture with a thinking mode that can emit reasoning traces before its response [Gem26]. The third approach is reinforcement-learning reasoning: DeepSeek-R1 is trained via large-scale RL with Group Relative Policy Optimization (GRPO), which develops internal Chain-of-Thought (CoT) capabilities, and it emits latent deliberation traces encapsulated in `<think>` tokens before each answer [Dee25]. The 32B variant evaluated here (`deepseek-r1:32b`) is a distilled model fine-tuned on reasoning traces from the full model [Dee25].

Evaluating these models requires quantitative, execution-based metrics rather than subjective

conversational scoring. The benchmark assesses each model across a 30-question dataset using two execution-based metrics:

1. **AST Syntax Validation Pass Rate (%):** measures whether LLM-generated equations conform strictly to SIRcraft's plain-text grammar without manual correction. Equations isolated by the gateway's extraction service are parsed by a recursive-descent AST validator designed to accept the same surface syntax as SIRcraft's ANTLR grammar [Gre25], isolating superficial formatting failures—such as raw LaTeX markup or illegal characters—from the underlying mathematical logic [HSH+25].
2. **Negative Boundary Refusal Precision (%):** quantifies how reliably candidate models identify and decline out-of-scope feature requests.

These two execution-based metrics provide the primary quantitative results; a broader multi-dimensional rubric covering factual grounding, response completeness, and UI actionability is presented in Chapter 5.

Chapter 3

Concept

3.1 Design Overview

This chapter presents the conceptual design of the SIRcraft RAG assistant. The system combines a domain-grounded retrieval pipeline, a locally orchestrated generation stage, and a post-processing validation gateway that enforces SIRcraft's equation grammar, addressing the four research questions introduced in Chapter 1:

- **RQ1 (Knowledge Base Engineering):** A hand-curated corpus of 179 self-contained chunks, drawn from SIRcraft thesis chapters, video tutorial transcripts, built-in example model specifications, and application manual entries, is embedded with `nomic-embed-text` and indexed into a ChromaDB vector store.
- **RQ2 (RAG Grounding and Operational Guidance):** A retrieval-augmented generation pipeline combines dense vector search over the knowledge base with locally hosted open-weights language models to provide grounded software guidance through a chat interface.
- **RQ3 (AST Validation):** A post-processing gateway extracts candidate equations from model responses, strips LaTeX and formatting artifacts, and validates them against a recursive-descent parser designed to accept the same surface syntax as SIRcraft's ANTLR `Expression.g4` grammar.
- **RQ4 (Negative Boundary Precision):** The system prompt enforces refusal awareness through few-shot examples of unsupported feature requests and a fixed refusal sentence, keeping boundary behavior uniform across models.

The system is deployed across two University of Bremen servers, as illustrated in Figure 3.1. An *Application & Web Server* hosts the Flutter web frontend (served through Nginx), the FastAPI application gateway, and the ChromaDB vector store. A dedicated *AI GPU Host* (NVIDIA RTX 5090, network address `172.22.255.248`) runs Ollama, serving both the `nomic-embed-text` embedding model and the four evaluated language models (`gemma2:9b`, `gemma4:26b`, `gemma2:27b`,

and `deepseek-r1:32b`).[1] All inference, vector storage, and application logic remain inside the institutional boundary.

The remainder of this chapter details each component in turn: Section 3.2 covers the knowledge base pipeline; Section 3.3 covers prompt construction and the structured output contract; Section 3.4 covers the post-processing gateway; and Section 3.5 covers the Flutter interface and the deployment topology.

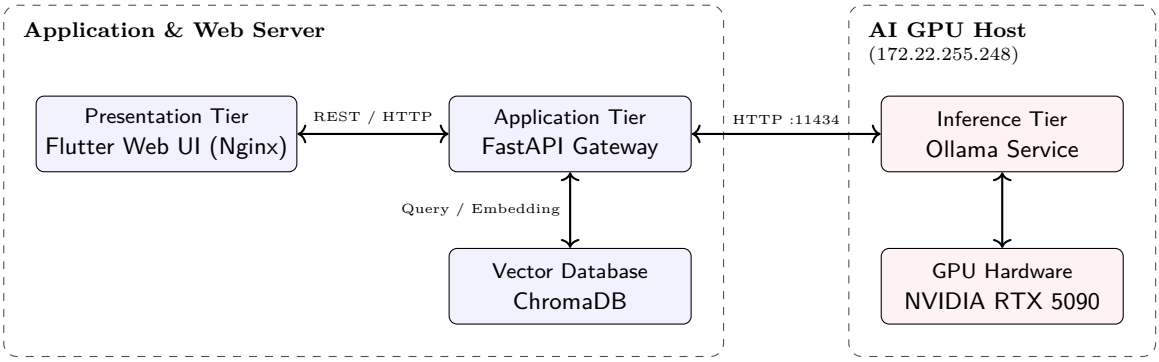


Figure 3.1 Deployment topology and three-tier architecture of the SIRcraft RAG Assistant across two University of Bremen servers.

3.2 Knowledge Base and Retrieval Design

The first design pillar of the SIRcraft RAG assistant is a domain-grounded knowledge base that provides the retrieval substrate for every user query. This section describes how the knowledge base was curated, normalized, chunked, embedded, and indexed for retrieval.

3.2.1 Source Curation

Unlike general-purpose RAG systems that scrape large corpora from the web, the SIRcraft knowledge base was curated manually by the thesis author. This decision reflects a deliberate trade-off. Automated ingestion maximizes coverage, but it applies generic splitting heuristics that are agnostic to the structure of the source material. In a mathematical modelling domain, such heuristics are risky: a fixed-length or sentence-based splitter can truncate an equation mid-expression, separate a transition definition from its parameter table, or cut a piecewise condition away from its fallback branch. Any of these failures would leave the retrieved context corrupted and cause the language model to generate syntactically broken or semantically incomplete answers. Manual curation avoids this class of failure by ensuring that every chunk boundary respects the

[1] Registered in Ollama as `gemma2`, which resolves to the 9B variant.

logical structure of the source text. Because the target domain is a single software platform with well-bounded documentation, manual curation is both feasible and expected to improve retrieval precision.

After gathering the source material, the author reviewed each entry, removed redundant chunks, and retained only those that contribute distinct retrieval value. The resulting corpus comprises approximately 70 pages of self-contained chunks—179 in total—drawn from four source categories.

SIRcraft thesis documentation.

The chapters of Grebe [Gre25] form the core reference for the platform, covering its motivation, domain foundations, related work, and design rationale.

Built-in example specifications.

SIRcraft ships with ten built-in example models that demonstrate common epidemiological structures, ranging from simple SIR models to multi-population vector-borne and vaccination scenarios. For each example, the author manually extracted every transition equation, every variable definition, and the corresponding auto-generated ODE system directly from the application, and wrote them into the chunk text. This ensures that the retrieval corpus captures the exact equations a user would see inside SIRcraft, rather than a paraphrase that could introduce notation drift.

Video tutorial transcripts.

The official "An Introduction to SIRcraft" tutorial series provides eight video transcripts covering model construction, parameter configuration, model distribution, exporting, vector-borne modeling, editor features, scenario comparison, and feedback submission.

Application manual entries.

Procedural documentation describing the graphical editor, equation syntax, transition and compartment configuration, and export workflows. In addition, the author manually consulted the application to identify and document important features that were not covered in the existing written sources.

3.2.2 Five-Field Chunk Schema

Each chunk follows a uniform five-field schema that separates retrieval metadata from content:

1. **Title:** a numbered identifier and a short descriptive phrase (e.g., *Chunk 4.3 — Model Components: Compartments, Transitions, and Variables*).
2. **Chunk type:** a descriptive label carried in the source text (e.g., "user guide", "technical scope"), which the ingestion pipeline maps to one of five canonical values (motivation, feature, guide, technical, general) stored in the chunk metadata.
3. **Source:** the document, chapter, section, and page reference, recorded in the chunk text for traceability.
4. **Use for:** example questions the chunk is designed to answer.
5. **Content:** the passage text itself.

The schema serves three design goals. First, the numbered title allows deterministic chunk identification and ordering. Second, the *chunk type* field supports future metadata filtering. Third, the *use for* field acts as a paraphrase anchor: because the ingestion pipeline embeds the entire chunk (including the *use for* line) into a single vector, user queries that resemble the example questions are naturally drawn toward the matching chunk.

A representative chunk is shown below:

> **Chunk 4.3 — Model Components: Compartments, Transitions, and Variables**
> Chunk type: user guide (inferred as guide)
> Source: SIRcraft thesis, Chapter 4, Section 4.2.1, page 15
> Use for: Questions such as "How do I build a model in SIRcraft?", "What are variables used for?", and "How do I add elements to the canvas?"
> Content: SIRcraft provides a no-code interactive interface for building models based on their typical graphical representation as directed graphs. It contains three types of components: compartments, transitions, and variables...

3.2.3 Normalization and Semantic Chunking

Before chunking, the raw source text passes through four normalization steps. Leading Byte Order Mark characters are stripped to remove encoding artifacts. Duplicated `Chunk type:Chunk type:` prefixes are collapsed into a single `Chunk type:` prefix. Lines that consist solely of structural remnants such as "chunk" or "chunk type" are removed. Finally, section headers are rewritten into a uniform format using two regex patterns:

```
r'(?m)^\s*(\d+\.\d+)\s*[---]\s*'         -> r'Chunk \1 - '
r'(?m)^\s*Chunk\s+(\d+\.\d+)\s*[---]\s*' -> r'Chunk \1 - '
```

The first pattern handles bare numeric headers such as `4.3 - Model Components`; the second normalizes headers that already start with "Chunk" but use inconsistent dashes or spacing.

Together, they ensure that every chunk begins with the exact pattern `Chunk X.Y -`, regardless of how the original document formatted the header.

Chunk boundaries are then determined by a positive lookahead regex rather than fixed token lengths:

```
r'(?m)(?=^\s*Chunk\s*\d+\.\d+\s*-)'
```

This pattern splits the document at every header boundary without consuming the header itself. Each candidate chunk is validated to confirm that its first line matches the header pattern before being retained. The result is 179 self-contained chunks, each corresponding to a single software feature, parameter definition, or GUI workflow.

The rationale for semantic boundary splitting over fixed-length chunking is retrieval precision: a fixed-length splitter would arbitrarily cut documentation mid-sentence, producing vectors that blend unrelated concepts. Semantic boundary splitting keeps each chunk topically coherent, which sharpens the embedding and improves retrieval accuracy.

3.2.4 Embedding Model Choice

Chunks are embedded using `nomic-embed-text` [NMDM25], an open-weights, Apache-2.0-licensed embedding model with an 8,192-token context window. Three requirements motivated this choice.

Long context.

The 8,192-token window exceeds the 512-token limit of earlier models such as Sentence-BERT [RG19, NMDM25], allowing full equation systems in the built-in example specifications to be embedded without truncation.

Task-specific prefixes.

The model uses `search_document:` for documents and `search_query:` for queries [NMDM25]. The ingestion pipeline embeds each chunk with the document prefix, while the retrieval gateway embeds each query with the query prefix, aligning both into a shared semantic subspace.

3.2.5 Vector Store and Retrieval Strategy

Embeddings are stored and queried using ChromaDB [Chr24], an in-process, persistent vector database. ChromaDB was selected over alternatives such as FAISS and Pinecone because it runs entirely in-process with no separate server, persists the vector index across application restarts,

and exposes a native Python API that integrates directly with the FastAPI backend. These properties align with the offline, data-sovereign deployment described in Section 3.5.

ChromaDB uses a Hierarchical Navigable Small World (HNSW) index for approximate nearest neighbor search, which provides logarithmic query-time scaling with respect to the number of indexed vectors [MY20]. The retrieval gateway returns the top $K = 7$ document chunks for each query, a value selected to fit the 16,384-token inference context window (set via Ollama's `num_ctx`) while leaving room for the system prompt, conversation history, and generated answer. Retrieval uses ChromaDB's default L2 distance metric.

3.3 Generation Design

3.3.1 Prompt Architecture

Every chat request is assembled from three parts and sent to Ollama as a native chat-format JSON payload: a fixed **system message**, a sliding window of the **ten most recent conversation messages** (representing up to five turns), and a **current user message** that prepends the freshly retrieved knowledge-base chunks to the user's new question. Rather than concatenating these into one string, the assistant passes them as a structured list of role-tagged messages so that the model can distinguish instructions from prior dialogue from the new question. Listing 3.1 shows the payload shape for a typical request (abridged).

Listing 3.1 Structure of the JSON payload sent to Ollama. The system message is abbreviated; only one user/assistant turn is shown.

```
{
  "model": "gemma4:26b",
  "messages": [
    {
      "role": "system",
      "content": "You are the SIRcraft Assistant ...
                  GRAMMAR REFERENCE: integers, decimals, scientific notation,
                  + - * / ^, comparisons, functions, piecewise { expr : cond, fallback }.
                  RULES: ground answers in the retrieved context, refuse out-of-scope
                  requests, output one JSON object with 'answer' first, then 'equations'
                  and 'suggestions'.
                  FEW-SHOT EXAMPLES: SIR rate transition, piecewise intervention,
                  unsupported feature."
    },
    {
      "role": "user",
      "content": "What is sircraft"
    },
    {
      "role": "assistant",
      "content": "SIRcraft is a no-code modeling platform ..."
    },
    {
      "role": "user",
      "content": "CONTEXT FROM KNOWLEDGE BASE:
                  --- [Source: Chunk 6.1] ---
                  SIRcraft is distributed as a standalone application ...

                  --- [Source: Chunk 6.2] ---
                  SIRcraft is fully supported on Linux ...

                  --- (five further chunks omitted) ---

                  USER QUESTION: How do I install SIRcraft on Linux?"
    }
  ],
  "stream": false,
  "options": {
    "num_ctx": 16384,
    "num_predict": 8192,
    "temperature": 0.1
  }
```

The three parts correspond directly to the message roles.

System message.

Passed once at the top of the list and never changes across requests. It carries the SIRcraft grammar reference, the behavioural rules, and the three few-shot examples described in Section 3.3.2. Because the rules and examples are fixed, the assistant uses a single canonical system prompt that is identical across every model in the evaluation set, keeping the model comparison in Chapter 5 fair.

Conversation history.

The history block contains the ten most recent user and assistant messages in alternating order (representing up to five conversational turns). The cap at ten messages reflects the shared context budget. Under typical conditions, seven retrieved chunks consume roughly 1,500–2,000 tokens, the system rules another 1,200–1,500, and each answer averages several hundred to a few thousand tokens. Ten messages is the largest window that leaves headroom for the system rules and retrieved context without displacing either.

Current user message.

The only part that changes between requests. It concatenates the labelled retrieved chunks, followed by the user's question, under a single `user` role. Each chunk is prefixed by a `-- [Source: <title>] --` header, keeping adjacent chunks visually separate and giving the model a stable handle for provenance reasoning. The header is stripped from the final answer by the post-processing stage (Section 3.4), so the user never sees internal chunk identifiers.

Inference options.

Each request carries an `options` object that configures the Ollama runtime. The two parameters that matter most are `num_ctx` (the context window size) and `num_predict` (the maximum number of tokens the model may generate), both discussed in Section 3.3.4.

3.3.2 System Rules

The system message is the fixed part of every request. It is passed once at the top of the message list and never changes between queries, so the same instructions reach every model in the evaluation set. Its contents fall into three blocks: a grammar reference, a set of behavioural rules, and three few-shot examples.

Grammar reference.

The first block defines the exact mathematical syntax that SIRcraft accepts: numeric formats (integers, decimals, scientific notation such as `1.5e-3`), arithmetic operators (`+`, `-`, `*`, `/`, `^`), comparison operators, and built-in functions (`sin`, `cos`, `sqrt`, `exp`, `ln`, `abs`, `max`, `min`). It also states the piecewise syntax in full, since this is the construct most frequently mis-generated by general-purpose models:

```
{ expr1 :  cond1, fallback }
```

Finally, the grammar reference prohibits three constructs that language models tend to produce by default: LaTeX markup (e.g., `\frac`, `\beta`), differential notation (e.g., `dS/dt`), and imperative `if/else` statements. These are the exact constructs that would cause the SIRcraft parser to reject an otherwise correct equation, so prohibiting them at the prompt level is the first line of defence for RQ3; the post-processing gateway described in Section 3.4 is the second.

Behavioural rules.

The second block contains ten numbered rules. Table 3.1 groups them by purpose and names the research question each group supports.

Purpose	**Rules**
Grounding	Rule 2: base answers strictly on the retrieved context and conversation history. Rule 8: if the answer is not in the context, reply with a fixed refusal sentence.
Refusal	Rule 8 doubles as the refusal rule for out-of-scope requests. The fixed sentence keeps refusal behaviour uniform across models.
Format	Rule 4: use Markdown headers, bold text, and bullet points. Rule 5: never emit internal metadata such as `[PRIMARY CHUNK]` or chunk numbers. Rule 6: never fabricate URLs.
Math syntax	Rule 7: use plain-text SIRcraft syntax, wrap inline variables in single backticks, use triple backticks for standalone equations, and keep the `syntax` field free of English labels.
Output contract	Rule 9: generate exactly three follow-up suggestions. Rule 10: output a single JSON object with `answer` first.

Table 3.1 Behavioural rules in the system message, grouped by purpose.

Three rules carry special weight. Rule 2 is the primary grounding instruction (RQ2): it keeps the model's parametric knowledge from overriding the retrieved documentation. Rule 8 is the refusal rule (RQ4): it instructs the model to decline out-of-scope requests rather than guess, using a single fixed sentence so that refusal behaviour is comparable across models. Rule 7 is the syntax rule (RQ3): it constrains generation to the subset of plain-text math that SIRcraft's parser accepts, complementing the post-processing gateway rather than replacing it.

Few-shot examples.

The third block contains three worked examples, one per behaviour:

1. **SIR rate transition.** A question about modelling the infection transition, answered with the canonical mass-action equation (`beta * S * I`). This establishes the basic shape of a well-formed answer.
2. **Piecewise intervention.** A question about reducing transmission after a fixed day, answered with the piecewise expression `{ beta * 0.5 :  t > T_start, beta }`. This is the most important of the three: piecewise logic is the construct that most reliably fails in unconstrained model output, and it is also what timed interventions in epidemiological models depend on.
3. **Unsupported feature.** A question about stochastic simulation, answered with the fixed refusal sentence from Rule 8. This shows that a refusal is a complete, acceptable response and that the model should not attempt a workaround outside SIRcraft's documented scope (RQ4).

Three examples — one per behaviour — cover the space of responses the evaluation set exercises. More would lengthen the system message at the cost of the retrieved context budget (Section 3.3.1); fewer would leave one behaviour unanchored.

One canonical prompt across all models.

The same system message is sent to `gemma2:9b`, `gemma4:26b`, `gemma2:27b`, and `deepseek-r1:32b`. It is not adapted per model, so any difference in behaviour between the four models must come from the model itself, not from a tailored prompt.

3.3.3 Structured Output Contract

The rule that requires a single JSON object is the interface between the language model and everything that follows it. The user never sees JSON directly, but the backend depends on it: the equation extraction service (Section 3.4.2) reads the `equations` array, the frontend renders follow-up chips from the `suggestions` field, and the equation validator runs on both the structured array and any equations the extraction service identifies in the Markdown answer. Without a fixed schema, every downstream stage would have to re-parse free-form Markdown, which is both fragile and ambiguous.

The three-field schema.

The contract defines exactly three fields, in this order:

```
{
  "answer":      "<markdown string>",
  "equations":   [ { "name": "<label>", "syntax": "<raw equation>" }, ... ],
  "suggestions": [ "<question 1>", "<question 2>", "<question 3>" ]
}
```

The `answer` field is the human-facing Markdown response; the `equations` field is an array of labelled plain-text equations as they would be typed into SIRcraft; the `suggestions` field is an array of exactly three follow-up questions. The split is deliberate: the Markdown is optimised for readability, while the `equations` array is optimised for machine processing, since the `syntax` values are exactly the strings the AST validator in Section 3.4 will later parse. If equations lived only inside the Markdown, the extraction service would have to guess which backticked fragments are real equations and which are explanatory code; the schema removes that ambiguity by construction.

Why `answer` must come first.

The ordering rule is not stylistic. Decoder-only language models generate tokens left to right, and JSON objects must be produced in the order their keys appear. If the model ran out of output budget partway through the object, whichever field came last would be truncated and lost. By requiring `answer` first, the prompt guarantees that the most important field is always present even when the `equations` or `suggestions` arrays are cut short — the same failure mode the Salvager (Section 3.4.1) exists to recover from.

Reasoning-model handling.

`deepseek-r1:32b` emits internal deliberation before its final answer, wrapped in `<think>...</think>` tags. The system message accommodates this: the model may produce a reasoning block first, provided it emits the complete JSON object after the closing tag. The reasoning block is stripped before parsing — both because it is not part of the schema and because it often contains draft equations that would confuse the extraction service if treated as real output (Section 3.4).

Contract versus reality.

During development testing, the contract was honoured most of the time but not universally. Three recurring deviations were documented: JSON wrapped in a Markdown code fence, fields nested under an alternate top-level key such as `response` or `markdown`, and reasoning-model outputs truncated before the closing brace when the token budget was exhausted mid-object. The recovery mechanisms that handle these deviations are described in Section 3.4.

3.3.4 Model Orchestration

The final design decision in the generation stage concerns how the request reaches the model and what computational constraints the model runs under.

Model selection.

All four evaluated models (`gemma2:9b`, `gemma4:26b`, `gemma2:27b`, and `deepseek-r1:32b`) are hosted on the same local Ollama instance. The frontend allows the user to switch between them at any point during a conversation, and each request carries the selected model name in its JSON body. If the name does not match one of the four configured identifiers, the backend falls back to the default model rather than rejecting the request, so a mismatch between frontend and backend does not cause a hard failure.

Deterministic generation.

Equation production is not a creative task: for a given question and a given retrieved context, there is essentially one correct plain-text formula, and any variation in operator or variable naming risks failing the AST validator. The sampling configuration therefore shifts the model strongly toward its most probable token at each step, in contrast to the higher-variance sampling that general-purpose chat assistants typically use.

Shared context budget.

Each request allocates the same 16,384-token context window and the same generation cap across all four models. The budget is shared between the fixed system message, the conversation history, and the retrieved chunks, and it determined both the value of $K = 7$ in Section 3.2.5 and the ten-message history cap in Section 3.3.1. Because the budget is fixed across models, the evaluation in Chapter 5 compares the four models under identical memory and latency conditions.

Fault tolerance.

Transient failures from the local inference server — timeouts or temporary unavailability, typically occurring when a large model is loading into GPU memory for the first time — are retried with exponential backoff. If all retries fail, the request returns a structured error message to the frontend rather than a partial response, so the user always receives either a complete answer or a clear failure.

3.4 Post-Processing and AST Validation

The generation stage produces a JSON object, or something close to one, on every request. Everything that follows ensures that the user receives a coherent answer and that any equations it contains are syntactically valid for SIRcraft. This section describes the post-processing pipeline in two parts: recovery and equation extraction (Sections 3.4.1 and 3.4.2), followed by grammar validation (Section 3.4.3). Failures at the validation stage are surfaced to the user as guardrail warnings rather than silently dropped. Figure 3.2 shows the resulting pipeline.

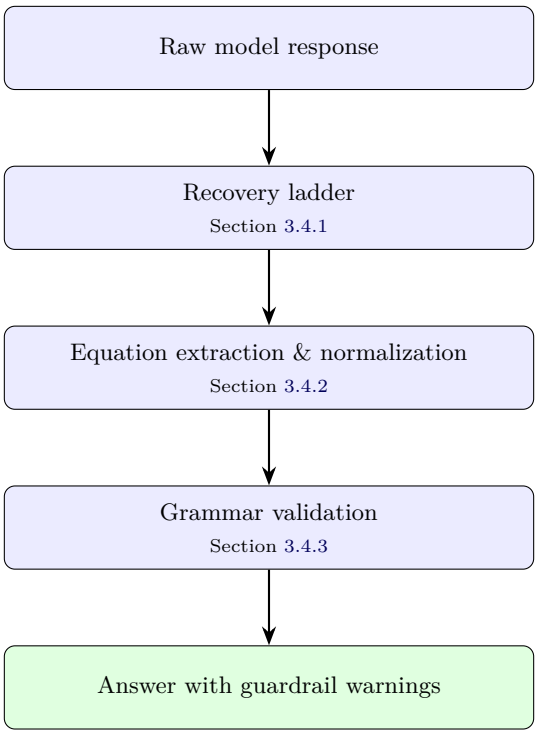


Figure 3.2 The post-processing pipeline. Each stage feeds the next; failures at the validation stage are surfaced to the user as guardrail warnings rather than silently dropped.

3.4.1 JSON Recovery: The Salvager

Despite the contract described in Section 3.3.3, the model does not always return well-formed JSON. The three deviation classes listed there were each observed during development and each would break a naive `json.loads` call.

The assistant handles this with an escalating recovery ladder. Each rung is attempted only if the previous one fails, and each is progressively less strict about the shape of the input. Figure 3.3 shows the four rungs.

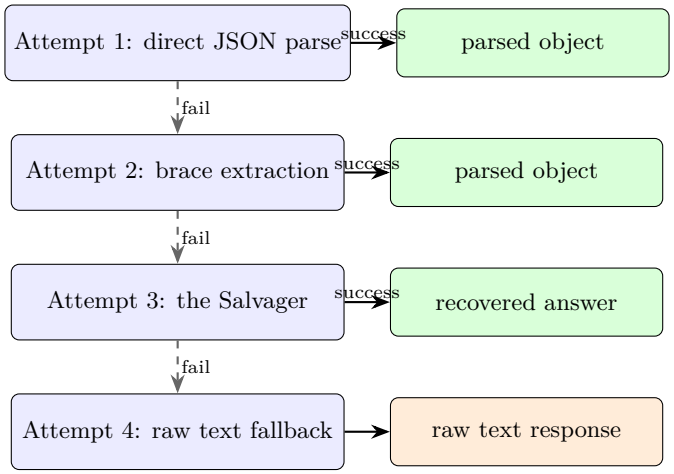


Figure 3.3 The escalating recovery ladder. Each rung is attempted only if the previous one fails. Solid arrows mark success; dashed arrows mark escalation to the next rung.

Attempt 1: direct parse.

The raw response is stripped of any leading Markdown fence (e.g., ```` ```json ````) and trailing closing fence, then parsed with a standard JSON decoder. If the model honoured the contract, this succeeds and the parsed object proceeds directly to the sanitizer described below. This is the expected path for the majority of requests.

Attempt 2: brace extraction.

If the direct parse fails, the next step assumes the JSON object exists somewhere inside the response but is surrounded by noise. The recovery logic scans for the first opening brace and the last closing brace and treats everything between them as a candidate. If the candidate parses and contains an answer-like key (`answer`, `response`, or `markdown`), it is accepted. Otherwise, the extractor tries all brace-delimited substrings, longest first.

Attempt 3: the Salvager.

If no candidate block parses — typically because the response is truncated and has no closing brace — the Salvager extracts only the `answer` field, using a regular expression that matches the key followed by its string value. It operates on both the raw response and the reasoning block, since some reasoning models emit the JSON payload inside their thinking trace. The extracted value is unescaped and passed through the metadata scrubber. The cost is that the structured metadata is lost: the `equations` array is returned empty and the `suggestions` array is replaced with a fixed default. The answer itself is preserved, and the pipeline appends a short note informing the user that the output was recovered from a malformed reply.

Attempt 4: raw text fallback.

If even the Salvager finds nothing — when the model returned only prose, only reasoning, or nothing at all — the pipeline returns the response body as-is after stripping internal reasoning markers. This rung guarantees that the pipeline always produces some response rather than an empty screen.

Sanitization of the recovered object.

Once a JSON object has been parsed by Attempt 1 or 2, it passes through a sanitization step before its fields are used. The sanitizer exists because models deviate from the schema in ways that do not break JSON syntax but do change the shape of the data: the answer text may be nested under an alternate key such as `response` or `markdown`, the suggestions may be labelled `follow_up_questions` instead of `suggestions`, or the entire payload may be wrapped in a sub-object. The sanitizer normalizes these variations into the canonical three-field form so that downstream stages always see the same structure. Its behaviour is described in Chapter 4. The next subsection describes how equations are extracted from the recovered answer text and from the structured `equations` array before they reach the AST validator.

3.4.2 Equation Extraction and Normalization

Once a JSON object has been recovered, the pipeline identifies every equation it contains and normalizes each into the plain-text form SIRcraft's parser expects.

Two sources of equations.

Equations can reach the pipeline from two places. The first is the structured `equations` array, where each entry carries a label and a raw syntax string. The second is the Markdown `answer` text, where equations may appear inside code fences, inside single backticks, or as standalone lines. Both sources are processed and merged at the end. The structured source is preferred because it was produced under the schema's discipline; the text source exists because models sometimes write additional equations into the prose that they forgot to include in the array.

Extraction from answer text.

Extraction from prose is a filtering problem: the text mixes real equations with Markdown headers, file paths, URLs, and ordinary English. The heuristic collects candidates from three places — triple-backticked code blocks, single backticks containing a mathematical operator or piecewise colon, and standalone lines with at least one operator — and passes each through a

rejection filter that discards URLs, file extensions, version numbers, transition arrow notations such as `S -> I`, English phrases with two capitalised words in a row, and placeholder templates such as `{ expression1 : condition1, fallback }`. The filter is deliberately conservative: it is better to miss a valid equation than to accept a sentence fragment as an equation.

The normalization pipeline.

Every equation — from the structured array or extracted from prose — passes through the same pipeline before validation:

1. **LaTeX stripping.** Commands such as `\times` and `\cdot` are replaced with `*`, and Greek-symbol macros such as `\beta` are converted to their plain-text names.
2. **ODE notation removal.** Differential markers such as `dS/dt = ...` and `d/dt S(t) = ...` are stripped, keeping only the right-hand side.
3. **Label stripping.** Leading labels such as `Equation:` or `Rate:` are removed.
4. **Piecewise boolean normalization.** Inside a piecewise condition, some models join comparisons with `*` rather than `&&`; the normalizer rewrites the condition with the correct boolean operator.
5. **Vector-notation normalization.** Compartment names in vector-borne models are rewritten to the canonical lowercase convention (`S_H` → `S_h`, `I_m` → `I_v`, and so on).
6. **Whitespace and punctuation cleanup.** Repeated spaces and newlines are collapsed, and trailing punctuation is removed.

Two filters run before normalization: empty brace expressions such as `{ }`, and placeholder templates matching the example syntax shown in the system prompt. Table 3.2 shows the effect of the pipeline on representative inputs.

Deduplication.

The structured and extracted lists are merged and deduplicated on the normalized syntax string. The structured form wins the tie because it carries a label that the extracted form lacks, and the label is what the frontend shows the user. Every equation in the merged list is then passed to the grammar validator described in the next subsection.

3.4.3 Grammar Validation

SIRcraft defines its equation syntax in an ANTLR 4 grammar file named `Expression.g4` [Gre25, Par13]. The grammar accepts arithmetic operators, comparison operators, boolean connectives, a fixed set of named functions, and a piecewise construct delimited by curly braces; SIRcraft parses user-entered equations against this grammar and lowers the resulting parse tree into an

Model output	After normalization	Step
`\beta \times S \cdot I / N`	`beta * S * I / N`	LaTeX
`dS/dt = -beta * S * I / N`	`-beta * S * I / N`	ODE
`Equation: beta * S`	`beta * S`	Label
`{ 0.5 : t > 10 * t < 20, 1 }`	`{ 0.5 : t > 10 && t < 20, 1 }`	Piecewise
`I_m * S_H / N_H`	`I_v * S_h / N_h`	Vector
`beta * S .`	`beta * S`	Cleanup
`{ expression1 : condition1, fallback }`	*(rejected before normalization)*	Placeholder

Table 3.2 Effect of the normalization pipeline on representative inputs. The last row is filtered out before normalization because it matches the placeholder template from the system prompt.

abstract syntax tree (AST) for numerical evaluation [Gre25]. The assistant needs the same guarantee—that every equation it shows the user will parse in SIRcraft—but it does not need to run ANTLR itself.

The validator is therefore a standalone recursive-descent parser written in pure Python. It is designed to accept the same surface syntax as `Expression.g4` but is implemented independently. Three reasons motivated this choice: the generated ANTLR parser would add a runtime dependency (`antlr4-python3-runtime`) unrelated to the assistant's own logic; the generated files are pinned to the ANTLR version that produced them (4.13.1), so any regeneration on SIRcraft's side would require a matching update; and the assistant only needs a yes/no answer about syntactic validity, which does not require a full ANTLR parse tree.

Tokenizer.

The first stage turns a raw equation string into typed tokens across six classes:

```
SCIENTIFIC   decimal or scientific-notation numbers (762, 0.002, 1.5e-3)
VARIABLE     identifiers starting with a letter or underscore
OP2          two-character operators (==, !=, >=, <=, &&, ||)
OP1          single-character operators (+ - * / ^ ( ) { } : , < >)
WS           whitespace
MISMATCH     any character that does not match any of the above
```

The `MISMATCH` class catches LaTeX commands, unicode mathematical symbols, and other artifacts, and raises a `SyntaxError` that the validator turns into a structured error naming the offending character. This is what catches `\frac{...}{...}`, where the leading backslash is a `MISMATCH`.

Parser.

The parser is a recursive-descent implementation whose operator precedence matches `Expression.g4`. It processes tokens from loosest to tightest binding:

> disjunction (`||`) → conjunction (`&&`) → relation (`==`, `!=`, `<`, `<=`, `>`, `>=`) → additive (`+`, `-`) → multiplicative (`*`, `/`) → power (`^`) → unary (`+`, `-`) → primary (variables, numbers, function calls, parentheses, piecewise)

A parenthesized subexpression re-enters the top of the hierarchy, so grouping overrides precedence in the standard way.

Piecewise parsing.

Piecewise logic is the only construct outside the operator hierarchy. The parser detects an opening brace at the primary level and enters piecewise mode, expecting a sequence of `expression : condition ,` pairs followed by a fallback expression and a closing brace. Each condition is parsed as a boolean expression, so `{ beta * 0.5 :  t > 30, beta }` is accepted while `{ beta * 0.5 :  t, beta }` is rejected: the condition slot requires a comparison, not a bare variable. This matches `Expression.g4`'s `piecewiseIf` production, which requires a `disjunction` in the condition position.

Representative inputs and outcomes.

Table 3.3 shows the validator on five representative inputs. Two rows illustrate the two failure modes: LaTeX input is rejected by the tokenizer (the backslash is not a recognized token), while a piecewise construct with a bare variable in its condition slot is rejected by the parser. The remaining rows confirm that the validator accepts the same expression shapes that appear in the built-in example models.

Input	**Result**	**Reason**
`beta * S * I / N`	valid	Standard rate expression.
`\frac{\beta S I}{N}`	invalid	Tokenizer: `\` is not a valid character.
`{ beta * 0.5 :  t > 30, beta }`	valid	Piecewise with a comparison in the condition slot.
`beta_vb_ * (I_h / N_h) * S_v - mu_v * I_v`	valid	Nested parentheses and mixed operators.
`{ beta * 0.5 :  t, beta }`	invalid	Parser: piecewise condition is not a comparison.

Table 3.3 Validator behaviour on representative inputs.

Relationship to `Expression.g4`.

The validator is designed to accept the same surface syntax as `Expression.g4` but is not a byte-for-byte reimplementation: it was written independently as a recursive-descent parser rather than generated from the grammar file, and it builds different internal trees. The two are intended to be equivalent on the class of equations the assistant produces, but no formal proof of equivalence exists. The empirical evidence is limited to the equations in the knowledge base that the validator test covers, all of which the validator accepts; divergence on expressions outside that set cannot be excluded.

Two known divergences exist between the validator, the system prompt, and SIRcraft's evaluator:

The first is a case where the validator is laxer. Its function whitelist includes `max()`, `min()`, and `if()`, but SIRcraft's SymPy visitor implements only seven function signatures: `sin`, `cos`, `sqrt`, `ln`, `exp`, `abs`, and a two-argument `log(base, x)`. The prompt itself contributes to the discrepancy: its grammar reference (Section 3.3.2) advertises `max` and `min` to the language model, and a knowledge-base chunk uses `max(0, S)` as an example. An equation using any of these three functions therefore passes validation, appears valid in the assistant's response, and only fails when the user pastes it into SIRcraft. This is the known false acceptance, and it is documented in Chapter 5.

The second is a case where the validator is stricter. SIRcraft implements a two-argument `log(base, x)`, but neither the validator nor the system prompt supports it. A SIRcraft equation that uses `log` would therefore be rejected by the assistant even though SIRcraft itself evaluates it correctly. This is the known false rejection. It is the mirror image of the first divergence: in both cases, the assistant and SIRcraft disagree on what counts as a valid function call, and the disagreement propagates to the user as a guardrail warning.

Guardrail warnings.

Every equation in the merged list from Section 3.4.2 is passed through the validator, and the result is attached to the entry as a boolean flag and, if applicable, an error message. When one or more equations fail, the pipeline preserves them in the response with the `is_valid` flag set to false and appends a warning block listing each failure and its reason. The user sees both the offending equation and an explanation of why it will not parse, and can decide whether to correct it or ask a follow-up.

3.5 Frontend and Deployment

The preceding sections described the retrieval, generation, and validation stages of the SIRcraft RAG assistant. This section describes the user-facing client and its deployment. The client is a Flutter web application served through Nginx; the backend, vector store, and inference service run across two University of Bremen servers. Section 3.5.1 covers the client architecture; Section 3.5.2 covers the conversation surface; Section 3.5.3 covers packaging and deployment.

3.5.1 Chat Interface Architecture

The client is a browser-based Flutter application. Flutter was chosen for three reasons. First, it is cross-platform: the same Dart codebase compiles to web, mobile-ready apps, and Windows desktop applications. This also yields a single web bundle that runs in modern browsers, so users reach the assistant without installation or updates—the same zero-installation principle as SIRcraft's desktop packaging. Second, a single Dart codebase produces responsive desktop and mobile browser layouts through a breakpoint system that switches between an expanded sidebar and a collapsible drawer. Third, Flutter's reactive widget model matches the incremental, message-by-message nature of a chat interface. The client uses the Business Logic Component (BLoC) pattern via `flutter_bloc`. Instead of widgets calling the backend directly, the application separates presentation from logic into three layers (Figure 3.4): widgets render state, cubits produce state, and a single service layer performs network calls. This separation keeps widgets free of business logic, centralizes conversation rules for unit testing, provides a single source of truth for each concern, and decouples the HTTP contract from conversation rules.

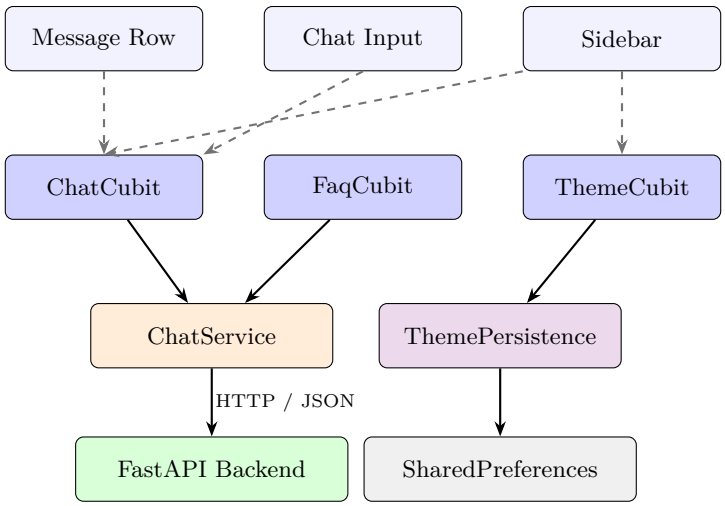


Figure 3.4 Layered client architecture. Widgets render state and dispatch actions to cubits (dashed); cubits perform the network and persistence calls (solid). Only the chat and FAQ paths reach the backend. The empty-state widget is omitted for clarity.

Three cubits each own one area of state. `ChatCubit` owns the conversation: messages, selected model, waiting/stopped flags, and error field. It exposes sending, clearing, model selection, stopping an in-progress reveal, and regenerating the last turn, and drives the typing simulation. `FaqCubit` fetches dynamic example questions for the empty state. `ThemeCubit` manages light/-dark/system theme and persists it across sessions. All three are provided at the application root.

Beneath the cubits, a thin stateless service layer wraps HTTP calls to the FastAPI backend. The service builds request URIs relative to the browser's current origin, with a fallback port for local development. It applies a request timeout, decodes JSON responses into plain maps, and converts non-success responses into structured exceptions. This layer is the only place that knows the backend's URL shape and response schema.

3.5.2 Conversation Presentation

The conversation surface is built from composable widgets, each rendering one part of a turn. Every turn is a message row with an avatar, header, body, and action bar. User turns are right-aligned plain text; assistant turns are left-aligned Markdown in a bordered card. The header shows the sender and, for assistant replies, the model that produced the response. The action bar offers copy on every turn and regenerate on the last turn.

Assistant replies are wrapped in a card containing the Markdown answer followed by up to three sections. The equations section lists extracted and validated equations with name, monospace syntax, and validity marker. The sources section lists retrieved chunks with title and excerpt. The suggestions section renders three follow-up questions as clickable chips. Each section appears only when its data is present.

The backend returns a single HTTP reply, so the client cannot stream tokens. To preserve perceived responsiveness, the conversation cubit reveals the answer character by character after the full response arrives. This is presentation-layer only. The stop button cancels the reveal and leaves partial text.

The sidebar (or drawer on smaller viewports) exposes the model selector, new-chat button, and theme toggle. The model selector lists four evaluated models with aliases and descriptions; the selected model is sent with each request. The theme toggle cycles light/dark/system and persists the choice.

Errors and confirmations use an overlay-based notification service rather than the built-in snackbar, which would be obscured by the floating input. The service supports error, info, warning, and success severities, each with distinct color, icon, and duration, and renders dismissible faded cards. New notifications replace the current one rather than queuing. Backend failures are mapped to friendly messages for timeout, refused connection, server error, and validation error.

3.5.3 Deployment and Data Sovereignty

The assistant is embedded into the SIRcraft project website using an iframe and is accessible at *https://sircraft.uni-bremen.de/sircraft-ai/*.

The system runs on two University of Bremen servers. The first, the *Application & Web Server*, hosts the Docker Compose stack containing the Flutter web frontend, the FastAPI gateway, and the persistent ChromaDB vector store. The second, the *AI GPU Host* (NVIDIA RTX 5090, network address `172.22.255.248`), runs Ollama, serving both the `nomic-embed-text` embedding model and the four evaluated language models. The application server reaches the inference host's Ollama endpoint over HTTP on port 11434. Neither server is reachable from the public internet; administrative access to each is gated behind a separate University of Bremen VPN, so an operator must establish two VPN sessions to inspect or redeploy both machines. End users reach the assistant through the SIRcraft project website, which serves the iframe wrapper and, through it, the application server.

The application server runs two services plus one named volume. The `backend` service packages the FastAPI application and ChromaDB client, exposes port 8000, and mounts the ChromaDB persistence directory as a named Docker volume so the vector index survives restarts. Its environment variables configure the Ollama base URL, default model, local embedding model, and allowed CORS origins. The `frontend` service is a multi-stage build: the first stage compiles the Flutter web application, and the second copies the output into an Nginx image served on port 80. Both services share a single Docker bridge network, so the frontend reaches the backend by service name. Inside the frontend container, Nginx serves the compiled Flutter bundle as static files and forwards API requests to the backend.

Every stage of the pipeline—embedding, retrieval, generation, and validation—executes locally on the two servers. No request leaves the university network, and no external API key is required. This keeps preliminary models, unpublished datasets, and institutional research material inside the university's data-governance boundary, satisfying the data-sovereignty requirement.

Chapter 4 describes how these decisions are realised in code, and Chapter 5 evaluates the system against the four research questions.

Chapter 4

Implementation

Chapter 3 described the design of the SIRcraft RAG assistant at the conceptual level: which components exist, why each was chosen, and how they relate to one another. This chapter describes how that design was realised in code. From Section 4.2 onwards it covers the same components as Chapter 3, following the same progression, with the generation and post-processing stages presented under a single section and the frontend and deployment stages split into two. Section 4.1 first records the software environment and dependency versions. Section 4.2 covers the FastAPI application and the knowledge base ingestion pipeline. Section 4.3 covers the server-side pipeline: embedding, retrieval, generation, and equation validation. Section 4.4 covers the Flutter client. Section 4.5 covers containerisation, the Nginx reverse proxy, and the test suite.

Throughout this chapter, code is referenced by the file and function that contains it rather than reproduced in full. Only the excerpts that carry non-obvious logic are shown as listings; the remainder is described in prose and left for the reader to consult in the accompanying repository.

4.1 Development Environment

The system was developed and deployed against a small, pinned set of software dependencies. Pinning matters for a thesis artefact because the evaluation in Chapter 5 reports numerical results that depend on specific library versions and specific model checkpoints; reproducing those results requires the same versions to be available. This section records what was used and why it was chosen.

Backend language and runtime.

The backend is written in Python and packaged as a FastAPI application. Python 3.11 is used inside the Docker image, matching the `python:3.11-slim` base image on which the deployed

artefact is built. Dependencies are installed into the image with `pip install -no-cache-dir -r requirements.txt`. The file is the single dependency declaration for the backend and mixes runtime and test packages; transitive dependencies are resolved by pip at image-build time and are not enumerated. Table 4.1 lists the direct Python dependencies declared in that file.

Package	Version
FastAPI	0.115.8
Uvicorn	0.34.0
Pydantic	2.10.6
Pydantic Settings	2.7.1
HTTPX	0.28.1
ChromaDB	0.6.3
python-dotenv	1.0.1
Requests	2.32.3
AnyIO	4.14.1
typing-extensions	4.12.2
pytest	8.3.4
pytest-asyncio	0.24.0
pytest-mock	3.14.0

Table 4.1 Direct Python dependencies for the backend, taken from `requirements.txt`. The file covers runtime and test packages; transitive dependencies are not listed.

Frontend toolchain.

The frontend is a Flutter web application built against the stable Flutter channel. Its direct dependencies fall into four groups: state management (`flutter_bloc`, `equatable`), rendering (`flutter_markdown`, `markdown`), transport and persistence (`http`, `shared_preferences`, `url_launcher`), and serialisation (`dart_mappable` with its `dart_mappable_builder` code generator). The test tooling mirrors the backend's: `flutter_test` for the standard testing, `bloc_test` for cubit-level tests, and `mocktail` for HTTP and persistence mocks. The full dependency list is reproduced in Appendix A.

Inference runtime.

Local inference is served by Ollama 0.32.6, installed on the AI GPU host. Ollama exposes an HTTP API on port 11434 and manages the model files, the GPU offloading strategy, and the inference runtime itself. Two kinds of models are served. The embedding model is `nomic-embed-text:latest`, a 768-dimensional model with an 8,192-token context window. The

four evaluated language models are `gemma2:9b`, `gemma4:26b`, `gemma2:27b`, and `deepseek-r1:32b`.[1] The backend never loads these models itself; it communicates with Ollama over HTTP and treats the inference host as a black-box service. The consequences of this split for memory, latency, and data sovereignty were discussed in Section 3.5.3.

Containerisation.

Both the backend and the frontend are packaged as Docker images and orchestrated with Docker Compose. The backend image is built from `python:3.11-slim` and runs Uvicorn as the process manager. The frontend image is a multi-stage build: the first stage uses a community-maintained Flutter image to compile the web bundle, and the second stage copies the compiled output into an Nginx image that serves it on port 80. The Compose stack, the Nginx configuration, and the two-server deployment topology are described in Section 4.5.1.

Repository layout.

The source tree separates the backend, the frontend, and the operational scripts into distinct top-level directories. Listing 4.1 shows the layout at the depth that matters for the rest of the chapter.

The backend is the only component that knows how to reach Ollama.The frontend talks only to the backend. This keeps the inference endpoint invisible to the browser, which is what makes the data-sovereignty guarantee of Chapter 3 hold end-to-end: even a client that inspected every network request would see only the university application server, never the GPU host.

The next section describes the backend application itself and the ingestion pipeline that populates its vector store.

[1] Registered in Ollama as `gemma2`, which resolves to the 9B variant.

Listing 4.1 Top-level repository layout showing key backend and frontend components.

```
sircraft_assistant/
 backend/                          FastAPI application and knowledge base
  |- main.py                       Application entry point
  |- ingest.py                     Knowledge-base ingestion CLI
  |- mock_ollama.py                Local mock of the Ollama HTTP API
  |- requirements.txt
  |- Dockerfile
  |- app/
  |  |- api/routes.py              HTTP endpoints (/chat, /faq, /health)
  |  |- core/
  |  |  |- config.py               Pydantic settings
  |  |  |- kb_processing.py        Text normalisation and chunk splitting
  |  |  `- error_handling.py       Structured errors and retry logic
  |  |- models/schemas.py          Request and response schemas
  |  `- services/
  |     |- embeddings.py           Ollama embedding function
  |     |- retriever_service.py    ChromaDB retrieval
  |     |- prompt_service.py       System prompt and message assembly
  |     |- rag_service.py          Retrieval, generation, and JSON recovery
  |     |- equation_service.py     Equation extraction and normalisation
  |     `- validator_service.py    Recursive-descent syntax validator
  |- tests/
  |  |- test_ast_validator.py      Validator acceptance and rejection cases
  |  |- test_equation_service.py   Extraction and normalisation
  |  |- test_routes.py             Endpoint contract tests
  |  |- test_rag_service.py        RAG orchestration with mocked Ollama
  |  `- ...
  `- scripts/normalize_kb.py       Standalone KB normalisation CLI

 frontend/                         Flutter web client
  |- pubspec.yaml
  |- Dockerfile                    Multi-stage build (Flutter -> Nginx)
  |- nginx.conf                    SPA fallback and API reverse proxy
  |- lib/
  |  |- main.dart                  Application root and BLoC providers
  |  |- logic/
  |  |  |- cubits/                 ChatCubit, FaqCubit, ThemeCubit
  |  |  `- states/                 State classes and mappers
  |  |- models/                    Domain models (ChatMessage)
  |  |- screens/                   MainDashboard layout
  |  |- services/                  HTTP client and notifications
  |  |- theme/                     Material 3 theming
  |  |- widgets/                   Composable UI components
  |  `- utils/                     Error mapping, responsive helpers
  |- test/                         Cubit, service, and widget tests
  `- web/                          Web entry point and manifest

 .github/workflows/                CI (Flutter web deployment to Pages)
 docker-compose.yml                Backend and frontend orchestration
```

4.2 Backend and Knowledge Base

This section describes the FastAPI application and the ingestion pipeline that populates the vector store it queries. It covers the application contract, the knowledge-base processing functions, and the ingestion script that ties them together.

4.2.1 Configuration and Endpoints

Configuration.

The configuration object in `backend/app/core/config.py` is a Pydantic `BaseSettings` subclass. It resolves the repository root at import time by climbing three directories from the file itself, so that the ChromaDB persistence directory and the knowledge-base file are located relative to the code rather than to the working directory. This makes the backend runnable from any directory, which matters because the same code executes inside the Docker container and on developer machines. Every deployment-specific value is read from an environment variable with a sensible default: the Ollama base URL defaults to the AI GPU host, the default model to `gemma4:26b`, the embedding model to `nomic-embed-text:latest`, and the retrieval depth to `TOP_K=7`, the value discussed in Section 3.2.5. The settings class reads a `.env` file if one is present, with `extra="ignore"` so that unrelated variables do not cause a validation error.

Application setup and schemas.

The backend starts from `backend/main.py`, which instantiates the FastAPI application, attaches the CORS middleware, and mounts the router that carries the three endpoints. The CORS middleware is enabled with `allow_credentials=False`, because the assistant is stateless and has no cookies or ambient authentication for the browser to send credentials to. The three endpoints in `backend/app/api/routes.py` are thin wrappers that delegate to the RAG service; any uncaught exception is converted into an HTTP 500 response with a generic message. The public contract is defined by four top-level Pydantic schemas in `backend/app/models/schemas.py` — one request and one response for the chat endpoint, plus one response for each of the auxiliary endpoints — with field names written in `snake_case` internally but aliased to their public names.

4.2.2 Ingestion and Indexing

Pipeline shape.

The knowledge-base pipeline is implemented as two pure text-processing functions in `backend/app/core/kb_processing.py` and one orchestration script in `backend/ingest.py`, which is never imported by the running application. The functions implement the normalisation and splitting design described in Section 3.2.3, with one implementation detail added on the splitting side: before accepting each candidate, the splitter checks that its first line matches the header pattern, which discards the leading material before the first header. The regex patterns for header standardisation start with `\s*`, which allows headers that were indented in the source material to be detected.

The ingestor and the parse step.

The ingestion pipeline is implemented as the `KnowledgeBaseIngestor` class in `backend/ingest.py`, invoked as a script. Its constructor instantiates the embedding function and the ChromaDB persistent client; the embedding function does not contact the inference host until the first chunk is embedded, so the parse step itself completes without any network I/O. The `parse_knowledge_base` method populates the five-field schema of Section 3.2.2 for each chunk. It derives the chunk identifier from the header by extracting the numeric part and replacing the dot with an underscore (`Chunk 4.3` → `chunk_4_3`), and it assigns the chunk type via the `_infer_type` helper. Duplicate identifiers are logged as warnings rather than aborting the parse.

Reset-and-upsert strategy.

Before upserting, the ingestor deletes the existing collection and recreates it with the same name and embedding function, making ingestion idempotent: rerunning the script does not accumulate duplicate chunks. The upserts are batched at ten chunks per call; because each chunk is embedded by a separate HTTP request to Ollama, batching reduces the number of ChromaDB transactions rather than the number of embedding calls. The final stage logs the total number of chunks in the collection, which the `/health` endpoint returns as `database_chunks`. For the current corpus this total is 179, the same figure reported in Section 3.2.1.

4.3 Retrieval and Generation

This section describes the code that implements the retrieval and generation stages of the pipeline. Section 4.3.1 covers the embedding function and the retrieval service; Section 4.3.2

covers the RAG service that assembles the request and the recovery logic that handles non-conforming model output. The design of these stages was described in Chapter 3; the discussion here is limited to implementation aspects the design chapter did not cover.

4.3.1 Embedding and Retrieval

The embedding function.

The embedding function in `backend/app/services/embeddings.py` is the class OllamaEmbeddingFunction, which subclasses ChromaDB's `EmbeddingFunction` so that ChromaDB can call it transparently when a collection is created or queried. It exposes two entry points: the abstract `__call__` method that ChromaDB uses when it embeds documents internally, and a named `embed_texts` method that the retrieval service calls directly when it wants to control the task-type prefix. Both methods compute the `search_document:` or `search_query:` prefix described in Section 3.2.4 and then delegate to a shared `_fetch_embeddings` helper. That helper sends one HTTP request per text to Ollama's `/api/embeddings` endpoint, with a three-attempt retry loop and a fixed five-second delay between attempts, so that each text is retried independently if the inference host is briefly unreachable. The helper is strictly sequential: a batch of n texts issues n sequential requests, so ingestion throughput scales linearly with corpus size.

The retrieval service.

The retrieval service in `backend/app/services/retriever_service.py` exposes a single asynchronous method, `retrieve_context`, which the RAG service calls once per request. It first embeds the question with the `RETRIEVAL_QUERY` task type, then queries the collection with the resulting vector and the configured top-K value. If either step raises—most often because the embedding request to Ollama times out—the method retries through the collection's own text-query path, which lets ChromaDB embed the question internally; that path uses the collection's default embedding prefix rather than the query prefix, so its ranking is not guaranteed to match the primary path, but it is preferred to returning nothing. If the retry also fails, the method returns empty document and metadata lists, which the RAG service treats as legitimate empty context rather than as an error.

4.3.2 Generation and Recovery

The RAG service.

The RAG service in `backend/app/services/rag_service.py` exposes a chat entry point, `get_chat_response`, which the `/chat` endpoint calls once per request. The method retrieves con-

text, builds the native chat-format message list, and calls the private `_generate_with_ollama` helper inside a retry loop. The model name is resolved against the configured list of available models before the loop begins; an unrecognised name falls back to the default model. The retry loop attempts up to three times, doubling a five-second delay, and retries only when the caught error string contains one of five transient-failure keywords: `429`, `503`, `timeout`, `exhausted`, and `unavailable`. If all attempts fail, or if the failure is not classified as transient, the method returns a structured error response rather than raising. Once generation succeeds, the method delegates equation extraction and validation to the equation service and returns a response payload containing the answer, retrieved chunks and source titles, validated equations, and follow-up suggestions.

The Salvager.

The third rung of the recovery ladder described in Section 3.4.1 is implemented inside _generate_with_ollama. It runs only after the strict parse and the brace-extraction scan have both failed, abandoning the goal of recovering a valid JSON object and extracting only the `answer` field:

Listing 4.2 The regular expression that locates the `answer` field when full JSON parsing fails.

```
answer_matches = list(re.finditer(r'"answer":\s*"(.*)', clean_text, re.DOTALL))
```

The pattern is applied to the response text first and to the reasoning block second. The typical case is a response truncated mid-object: the model has emitted the complete answer text and several complete equation entries, but has run out of output budget before closing the array. A response of this shape might look like the following. The example is constructed for illustration; line breaks are shown for readability.

```
{"answer": "To model a lockdown, create T_start = 30 and use
the piecewise equation { beta * 0.5 : t > T_start, beta }
in the infection transition.", "equations": [{"name":
"Lockdown", "syntax": "{ beta * 0.5 : t > T_start, beta }"},
{"name": "Infection", "syntax": "beta * S * I / N"}, {"name":
"Recovery", "syntax": "gamma * I"}, {"name": "Vaccination",
"syntax": "nu * S"}, {"name": "Hospitalisation", "syntax":
```

The first regular expression matches the `answer` key and captures everything after the opening quote of its value. The second regular expression trims the captured text at the first unescaped closing quote followed by a comma or closing brace, which in this case is the quote that terminates the answer value. The resulting string is passed to the metadata scrubber. If no match is found on either source, the method falls through to the fourth rung, which scrubs the response text and returns it.

Sanitization and scrubbing.

Two helpers process a recovered response before it reaches the equation service. `_sanitize_ollama_json` runs only on a response that parsed as JSON and corrects shape deviations the schema does not anticipate. `_scrub_metadata` runs on every recovery path and cleans the answer string itself, removing residual reasoning blocks, internal chunk and source references, and LaTeX markup, and collapsing runs of blank lines.

Together the two helpers turn a raw JSON response such as:

```
{"response": "<think>The user wants a lockdown rate.</think>
Use a piecewise transition. The rate becomes
\beta \times S \times 0.5."}
```

into the answer the user actually sees:

```
Use a piecewise transition. The rate becomes
beta * S * 0.5.
```

4.3.3 Equation Validation

The extraction service.

The equation service in `backend/app/services/equation_service.py` exposes a single static entry point, `extract_and_validate`, on the `EquationExtractionService` class. It takes the structured equations returned by the model and the answer text, applies the extraction and normalisation pipeline of Section 3.4.2 to both sources, and returns a tuple of the merged equation list and the answer text with any guardrail warning appended. Structured equations are appended to the running list before extracted equations are considered, so an extracted equation is added only when its normalised syntax is not already present. A module-level singleton, `equation_extraction_service`, is instantiated at import time.

The validator.

The validator in `backend/app/services/validator_service.py` is the `SIRcraftValidator` class; the recursive-descent parser is a private `_Parser` class that the validator instantiates once per input. The tokenizer is a single compiled regular expression built from six named groups, and the parser walks the resulting token list with the precedence hierarchy described in Section 3.4.3. The validator accepts a fixed list of nine function names: `sin`, `cos`, `sqrt`, `exp`, `ln`, `abs`, `max`, `min`, and `if`. This list overlaps with, but does not match, SIRcraft's evaluator, which implements `sin`, `cos`, `sqrt`, `ln`, `exp`, `abs`, and a two-argument `log(base, x)`. The mismatches are the source of the divergences catalogued in Section 3.4.3: `max`, `min`, and `if` are accepted by the validator but

not evaluated by SIRcraft, and `log` is evaluated by SIRcraft but not accepted by the validator. A module-level singleton, `validator_service`, is instantiated at import time.

4.4 Frontend (User Interface)

This section shows the built Flutter client and names, for each figure, the widget file that renders it and the design point from Section 3.5.2 that the figure makes visible.

Landing view.

Figure 4.1 shows the empty state that replaces the conversation view on first open (`frontend/lib/widgets/chat/empty_state_widget.dart`). The gradient icon introduces the assistant, and the chips below are FAQ questions fetched from the knowledge base at startup, so the entry point always reflects the current corpus rather than a hard-coded list.

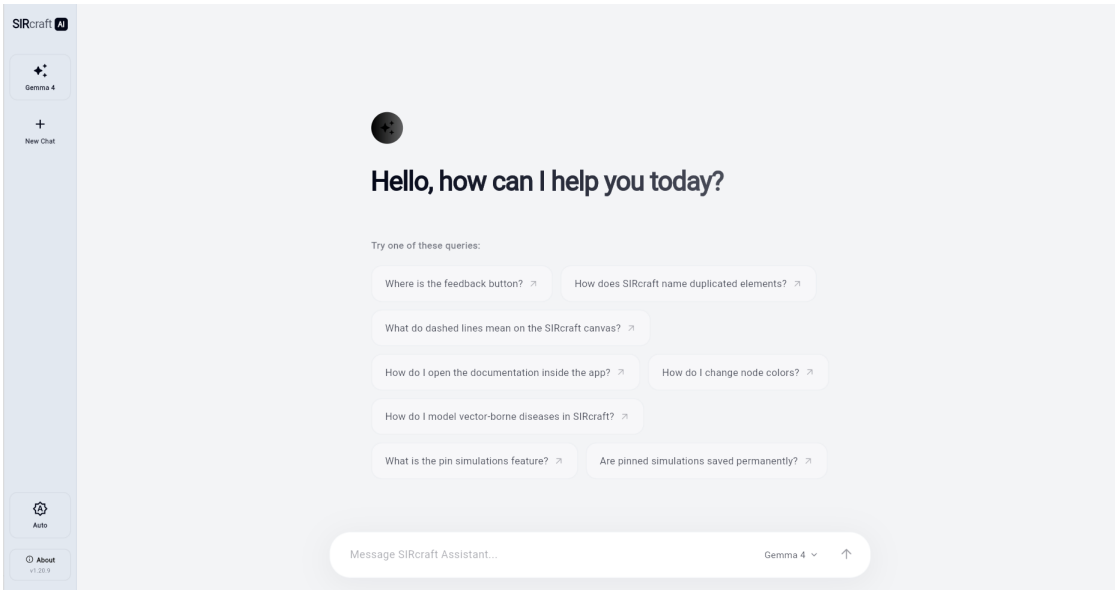


Figure 4.1 Landing view with FAQ chips.

Waiting state.

Figure 4.2 shows the state between submitting a question and receiving an answer: an assistant placeholder appears in the conversation with an animated dot indicator in place of the answer body (`frontend/lib/widgets/message_row/components/shared.dart`). The send arrow in

the input widget becomes a stop button for the duration of the wait, so the user can abandon the request before the response arrives.

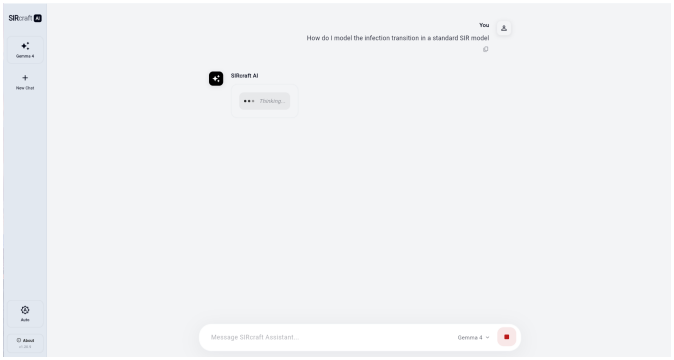


Figure 4.2 Waiting state during generation.

Completed reply.

Figure 4.3 shows a finished assistant turn, which carries four parts: a header with the model label that produced the response, a Markdown body, two collapsible panels below the body, and a row of suggestion chips (`frontend/lib/widgets/message_row/components/ai_card.dart`). The panels are collapsed by default so that a long answer is not immediately obscured by structured metadata; the chips offer three follow-up questions that feed back into the conversation.

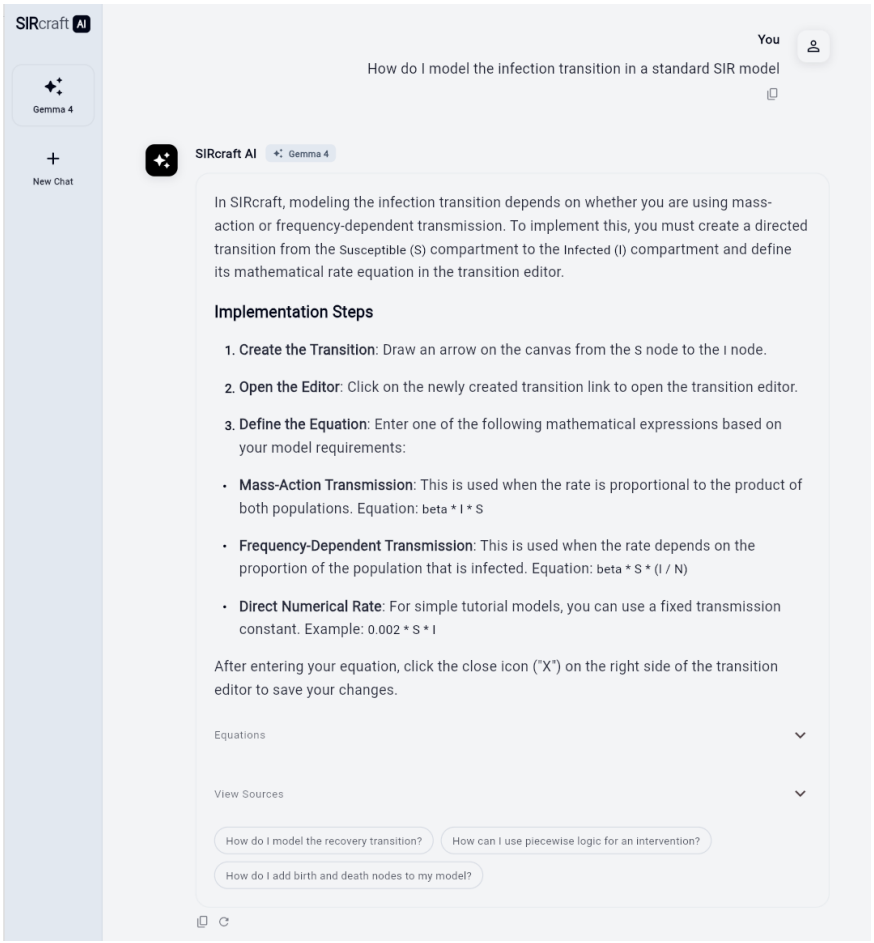


Figure 4.3 Completed assistant reply.

Expanded panels.

Figure 4.4 shows the same reply with both panels open (`frontend/lib/widgets/message_row/components/metadata_sections.dart`). The equations panel lists each equation the backend extracted and validated, with the label assigned by the model and a validity marker; the sources panel lists the retrieved chunks that grounded the answer, each with its title and full text so that a claim in the answer can be traced back to the passage that supports it.

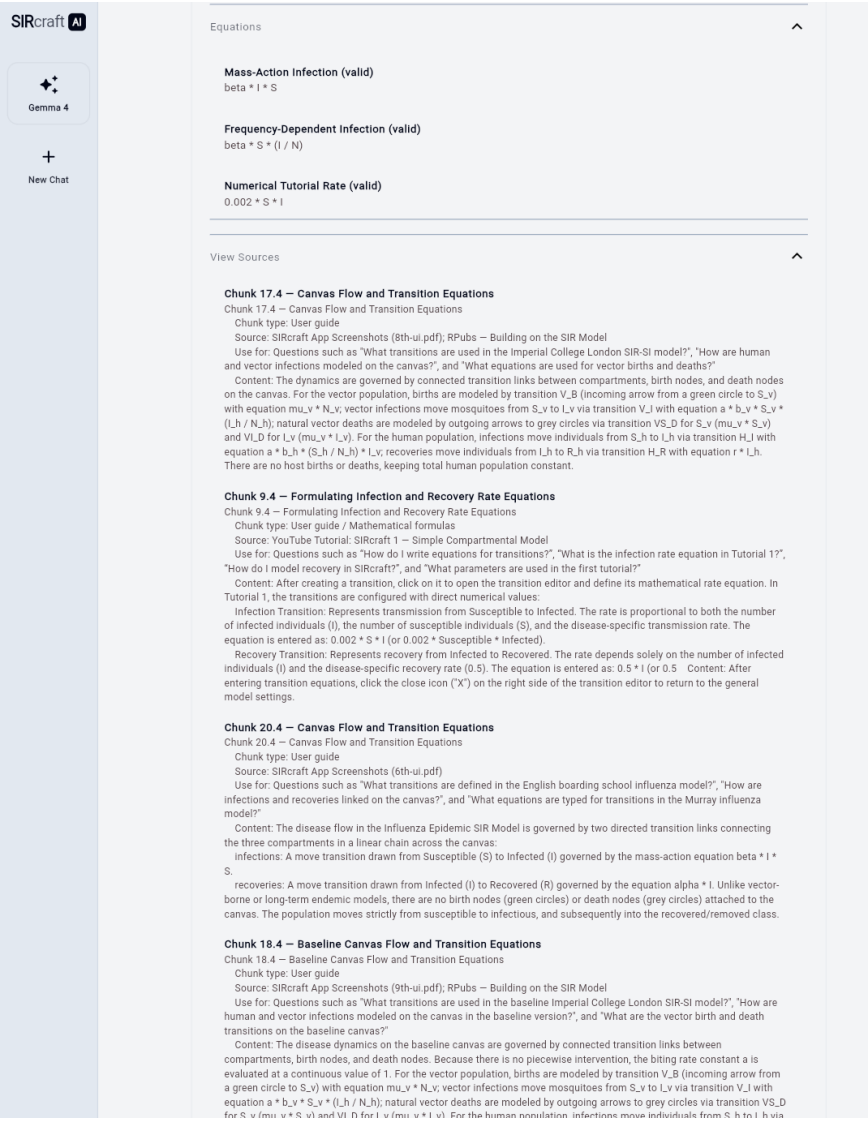


Figure 4.4 Equations and sources panels open.

Sidebar controls.

Figure 4.5 shows the two dropdown controls carried by the sidebar (`frontend/lib/widgets/sidebar/sidebar_widget.dart`). The model selector lists the four evaluated models with human-readable aliases and short descriptions; the theme toggle cycles between light, dark, and system modes, and the choice persists across sessions.

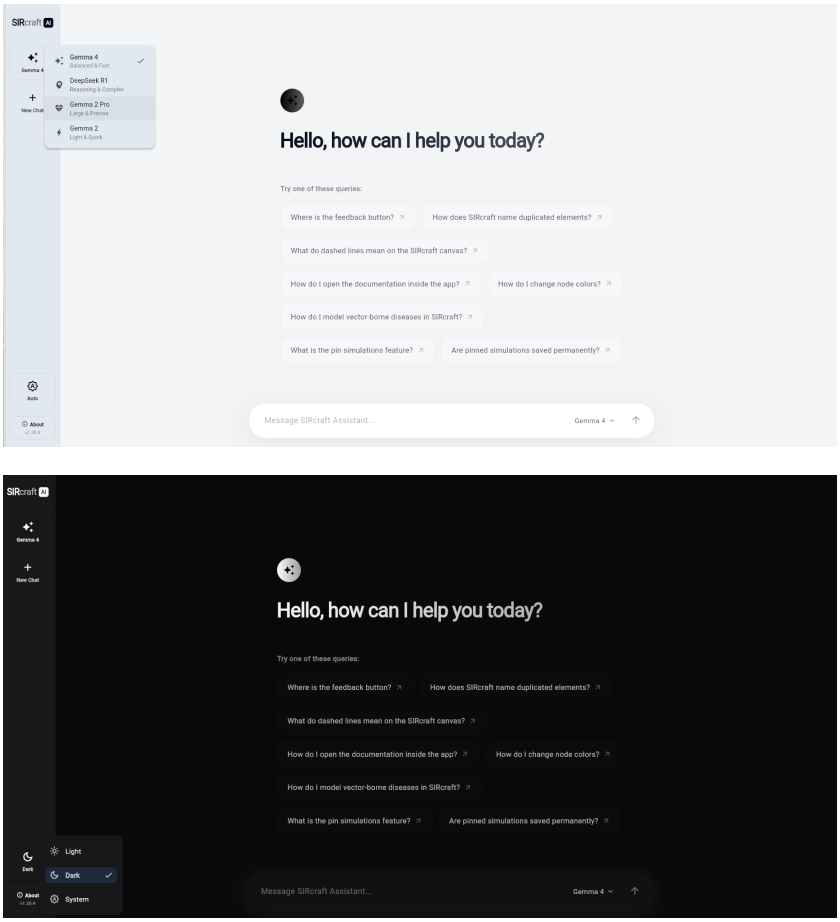


Figure 4.5 Sidebar dropdown controls. Top: the model selector. Bottom: the theme toggle.

4.5 Deployment and Tests

This section covers the artefacts that package and verify the system. The deployment topology was described in Section 3.5.3; the discussion here is limited to the code that realises it.

4.5.1 Containerisation

The Compose stack.

The deployment is defined by a single `docker-compose.yml` at the repository root, declaring two services, one named volume, and one bridge network. The `backend` service builds from `backend/Dockerfile`, exposes host port 8000, and mounts the named volume `chroma_data`

at `/app/backend/chroma_data`, the path the ingestion script writes to. Seven environment variables configure the container — server host and port, reload flag, Ollama base URL, default language model, embedding model, and allowed CORS origins — each with a default written into the Compose file itself, so the stack starts without an external `.env`. The `frontend` service builds from `frontend/Dockerfile`, exposes host port 80, and depends on the backend. Both services use `restart: unless-stopped` and are attached to a single bridge network named `sircraft-network`, which lets the frontend reach the backend by its service name rather than by host or IP.

The two images.

The backend image is built from `python:3.11-slim`. It installs `build-essential` as the only system dependency, installs the pinned Python packages from `backend/requirements.txt`, and copies only the `backend/` directory into the image, so the frontend and operational scripts are not shipped with the service. The container is started with Uvicorn bound to `0.0.0.0:8000`. The frontend image is a two-stage build: the first stage uses `ghcr.io/cirruslabs/flutter:stable` to fetch the Dart dependencies and compile the web bundle in release mode, and the second stage copies the compiled output into an `nginx:stable-alpine` image and replaces the default server configuration with `frontend/nginx.conf`. The final image therefore contains no Flutter toolchain, only the static bundle and the Nginx runtime.

4.5.2 Nginx Configuration

The frontend container serves the compiled Flutter bundle and the API through a single Nginx instance, configured in `frontend/nginx.conf`. Giving one process both responsibilities gives the browser a single origin for the application and the API, which removes the need for cross-origin negotiation in production. The CORS middleware described in Section 4.2.1 is therefore only relevant during development; in the deployed system every request is same-origin.

Serving the client.

Nginx listens on port 80 and serves the compiled bundle from the document root populated by the second Docker build stage. Every request that does not match a real file is rewritten to `index.html` rather than returning a 404, which lets Flutter's router handle the path on the client.

Forwarding the API.

Three `location ^~` blocks forward requests under `/chat`, `/faq`, and `/health` to the backend service by its Compose name on port 8000. The browser therefore never learns the backend's host or internal port; it only ever issues requests to the same origin that served the page. This is what keeps the inference host invisible to the client, as Section 3.5 described, and it means that moving the backend to a different machine or port requires a change in one configuration file rather than a change in the client.

Caching and compression.

The configuration splits responses into two classes. The HTML entry point and JSON documents are marked `no-store`, so a deployment is picked up immediately and no user runs a stale client against a newer backend. Static assets are marked `no-cache` with access logging disabled, so repeat page loads revalidate instead of re-downloading the whole bundle. Gzip is enabled above ten kilobytes for the text-based content types the bundle is made of.

4.5.3 Test Suite

Backend.

The backend tests run under pytest, with `pytest-asyncio` for the asynchronous service methods and `pytest-mock` for the HTTP-client and Ollama mocks. They are grouped by the component they exercise: knowledge-base processing and ingestion, configuration and error handling, the equation and validator services, the Ollama client and its JSON recovery, the prompt builder, the retrieval service, the RAG service with a mocked Ollama, the HTTP endpoint contracts, and one end-to-end test that starts the FastAPI application on a local port and issues real HTTP requests against it. The suite currently reports 67 test functions, all passing.

One of them, `test_kb_equations.py`, is worth singling out because it is the empirical basis for a claim made in Chapter 3: the file contains a manually compiled list of the equations that appear in the knowledge base, tagged by the chunk they come from, and passes each entry through the validator; every entry is accepted. That result is what allows Section 3.4.3 to describe the validator's surface compatibility with the SIRcraft grammar as verified rather than asserted. The list is not generated from the corpus at test time; it is a snapshot of the expressions that appear in the corpus and is maintained alongside it, so a new equation added to the knowledge base would need to be added to the list to remain covered.

Frontend.

The frontend tests run under `flutter_test`, with `bloc_test` for the cubit-level cases and `mocktail` for the HTTP and persistence mocks. They cover the three cubits, the chat service's URI construction and path normalisation, the notification overlay, the error-message mapper, and a widget-level smoke test that instantiates the root application and verifies it reaches the initial input prompt. The suite currently reports 30 tests, all passing. Together with the backend suite, these tests exercise every service and state class named in this chapter; the two test directories are the operational definition of what the chapter describes.

Chapter 5

Evaluation

This chapter evaluates the SIRcraft RAG assistant against the four research questions introduced in Chapter 1. *Sections 5.1 and 5.2 define the evaluation design and scoring rubrics, and Sections 5.3 and 5.4 present and interpret the results.*

5.1 Evaluation Design

5.1.1 Models and Benchmark Dataset

Four locally hosted open-weights models were evaluated, covering the model families introduced in Section 2.4.2:

- `gemma2:9b` — a lightweight dense instruction-tuned baseline;
- `gemma4:26b` — a mixture-of-experts instruction-tuned model, configured as the default;
- `gemma2:27b` — a high-capacity dense instruction-tuned model;
- `deepseek-r1:32b` — a chain-of-thought reasoning model.

All four were served by the same local Ollama instance (Section 3.5.3) and driven by the same canonical system prompt (Section 3.3.2), so the comparison in Section 5.3 is a comparison of models, not of prompts.

The benchmark consists of 30 questions distributed evenly across five functional categories, with four supported and two unsupported questions per category (20 supported, 10 unsupported overall). Table 5.1 summarizes the categories; the full list is given in Appendix B. The categories span the operational surface of the assistant, from installation workflows (Category 1) through canvas architecture (Category 2), equation grammar and AST constraints (Category 3), interactive visualizations (Category 4), and built-in example models (Category 5). Unsupported questions were written as plausible extensions that a user might reasonably expect to exist, so that a refusal reflects a genuine boundary decision rather than an obviously out-of-scope request.

Cat.	Focus	Supp.	Unsup.
1	Platform setup, packaging, and UI navigation	4	2
2	Compartmental canvas architecture and multi-species modeling	4	2
3	Mathematical grammar, AST syntax, and solvers	4	2
4	Dynamic scenarios, visualizations, and validation	4	2
5	Built-in benchmark case studies	4	2
Total		**20**	**10**

Table 5.1 Composition of the 30-question benchmark dataset by category. The full list is given in Appendix B.

5.1.2 Evaluation Protocol

All 30 questions were submitted to the assistant manually, one at a time, with each question isolated from the others: no conversation history was carried over, and the model selector was set before each submission. Isolation ensures that no model's earlier answer can influence a later one and that all four models score under identical conditions.

Equations were classified in two steps. The assistant's extraction pipeline first isolated every equation that appeared in a response, following the procedure described in Section 3.4.2. Each extracted equation was then validated against SIRcraft's own ANTLR grammar and function registry—the same grammar and the same registry a user would encounter if they pasted the equation into the application—by parsing it with the `Expression.g4` grammar and lowering the parse tree through the SymPy visitor that SIRcraft's simulator uses. Classification therefore reflects what SIRcraft itself accepts, not what the assistant's internal validator accepts.

Counts are computed on the raw extracted equations with no normalization: each equation is taken exactly as the extraction pipeline isolated it and passed unchanged to SIRcraft's grammar and function registry. The counts therefore include extraction artifacts the pipeline captured from the models' own grammar demonstrations (for example, placeholder-argument calls such as `sin(x)` and `cos(x)` that models emitted while illustrating the supported function set); these are themselves valid SIRcraft expressions and do not lower any model's pass rate. Assignment-prefixed forms such as `b_2 = {0 :  t <= 10, 0.02}` are captured by the extraction pipeline as equations and are counted as failures, because SIRcraft's grammar rejects the `=` prefix that the model emitted. The classification is reported per model in Section 5.3.4. All scoring was performed by the author.

5.1.3 Research Question to Metric Mapping

Table 5.2 connects each research question from Section 1.3.2 to the metric or exhibit that addresses it.

RQ	Focus	Metric or Exhibit
RQ1	Knowledge base engineering and vector retrieval	Category-level retrieval notes (Section 5.3.5)
RQ2	RAG grounding and operational guidance	Rubric A dimensions S1–S2; rubric dimension summary (Table 5.6)
RQ3	Post-retrieval AST validation	AST Syntax Validation Pass Rate; validation rates (Table 5.8)
RQ4	Negative boundary precision	Negative Boundary Refusal Precision; boundary adherence (Table 5.7)

Table 5.2 Mapping from each research question to the metric or exhibit that addresses it.

RQ2 through RQ4 are each assessed by a scalar metric and a supporting exhibit; RQ1 is assessed through the category-level retrieval notes recorded per question, because retrieval quality is not captured cleanly by a scalar. The exhibits referenced in the third column are presented in Section 5.3.

5.2 Scoring Rubrics and Metrics

5.2.1 Rubric A: Supported Questions

Every supported question was scored against a three-dimension rubric, each dimension rated on a three-point scale (0, 1, or 2). The dimensions are factual grounding, response completeness, and UI actionability including equation syntax. The three scores sum to a total between 0 and 6 per question; across the 20 supported questions, Rubric A contributes up to 120 of the 180 points available per model. Table 5.3 defines each score level.

The three dimensions isolate the distinct failure modes a supported question can elicit. S1 catches fabrication: a model that invents a button or parameter that does not exist in SIRcraft scores 0 regardless of how confident the rest of the answer sounds, because the user is sent looking for a feature that is not there. S2 catches omission: an answer that names the right concept but leaves out a required parameter or configuration step scores at most 1, because the user cannot complete the task from it. S3 catches executability and syntax validity: an answer whose prose is accurate and complete but whose equations fail in the SIRcraft parser scores at most 1, because the user still has to debug the equation before it runs.

Score	S1: Factual Grounding	S2: Response Completeness	S3: UI Actionability & Syntax
0	Fabricates buttons, parameters, or features not in SIRcraft documentation.	Misses critical steps or omits vital mathematical parameters required for execution.	Instructions fail or break if entered directly into the software.
1	Mostly accurate, but includes minor inaccuracies or imprecise terminology.	Explains high-level concepts but skips secondary parameters or configuration details.	Instructions work but require minor trial-and-error or debugging by the user.
2	Faithful to SIRcraft documentation, parameters, and GUI labels.	Exhaustive instructions covering all steps, parameters, and variable definitions.	Flawless, directly executable step-by-step guidance and valid SIRcraft syntax.

Table 5.3 Rubric A for supported questions. Each of the three dimensions is scored 0–2, giving a maximum of 6 points per question.

The three failure modes are independent — an answer can be factually correct but incomplete, complete but not executable, and so on — which is why the rubric scores them separately rather than as one combined judgment.

Rubric A is not applied to unsupported questions; for those, refusal correctness is the primary signal and requires a different set of dimensions. Rubric B, defined in Section 5.2.2, covers that case.

5.2.2 Rubric B: Unsupported Questions

Every unsupported question was scored against a three-dimension rubric that mirrors the structure of Rubric A but assesses different behaviors. The dimensions are refusal rule adherence, scope justification, and constructive alternative, each rated on a three-point scale (0, 1, or 2) and summed to a total between 0 and 6. Across the 10 unsupported questions, Rubric B contributes up to 60 of the 180 points available per model. Table 5.4 defines each score level.

The three dimensions isolate the distinct failure modes an unsupported question can elicit. U1 catches hallucination: a model that invents a workflow for a feature that does not exist scores 0, because it has actively misled the user rather than declined the request. U2 catches unjustified refusals: a model that refuses because it did not find the feature in its knowledge base scores at most 1, because the refusal is a retrieval gap rather than a boundary decision. U3 catches dead-end refusals: a model that declines correctly but offers nothing in return scores at most 1, because the user is left with a problem and no next step.

The distinction between a generic refusal (U1 = 1) and an informed refusal (U1 = 2) is the core

Score	U1: Refusal Rule Adherence	U2: Scope Justification	U3: Constructive Alternative
0	Hallucination: ignores the negative constraint and invents a fake workflow.	Gives no explanation or provides an incorrect mathematical rationale.	Provides misleading or mathematically impossible advice.
1	Generic refusal: states it has no information in its knowledge base.	Notes that the feature was not found in the documentation.	Suggests generic external tools without connecting back to SIRcraft.
2	Informed refusal: explicitly identifies the feature as unsupported by SIRcraft.	Explains the architectural boundary (e.g., deterministic ODE vs. stochastic/spatial).	Suggests valid SIRcraft workarounds (e.g., CSV export for external analysis).

Table 5.4 Rubric B for unsupported questions. Each of the three dimensions is scored 0–2, giving a maximum of 6 points per question.

of RQ4. A generic refusal indicates that the model could not find the answer; an informed refusal indicates that the model recognized a structural boundary in SIRcraft's design and declined on those grounds. The two are visibly different to a user reading the response, and only the second reflects the prompt-level boundary the system was built to enforce.

5.2.3 Execution-Based Metrics

The two execution-based metrics were computed per model by counting directly from the extracted-equation set described in Section 3.4.2. The first, the AST Syntax Validation Pass Rate, measures how many of the equations a model generated would parse in SIRcraft without manual correction. It is the ratio of equations that SIRcraft's parser accepted to the total number of equations the extraction pipeline captured:

$$\text{AST Pass Rate} = \frac{N_{\text{valid}}}{N_{\text{total}}} \times 100\% \tag{5.1}$$

The denominator counts every equation entry the extraction pipeline produced across all 30 questions, with no filtering or deduplication: equations that appear both in the structured array and in the prose, and brace-wrapped duplicates of an equation that also appears unwrapped, are each counted as separate entries, in keeping with the no-normalization method stated in Section 5.1.2.

The second metric, Negative Boundary Refusal Precision, measures how reliably a model declines unsupported requests. It is the ratio of correct refusals to the 10 unsupported questions:

$$\text{Boundary Precision} = \frac{N_{\text{refused}}}{10} \times 100\% \tag{5.2}$$

A refusal is counted as correct when the model declines to provide a workflow for the unsupported feature (that is, U1 ≥ 1 in Rubric B). The distinction between a generic refusal and an informed refusal does not enter this metric: both prevent the model from fabricating a workflow that does not exist. The quality difference between them is captured separately by U1 and is reported in Section 5.3.3.

Both metrics are reported per model in the results section, alongside the rubric-based scores.

5.3 Results

This section reports the scored results for the four evaluated models. Section 5.3.1 presents the aggregate scores, Section 5.3.2 rubric dimension summary, Section 5.3.3 the negative boundary behaviour, and Section 5.3.4 the equation syntax validation outcome. Section 5.3.5 closes the section with the category-level retrieval notes that support RQ1.

5.3.1 Overall Scores

Table 5.5 reports the total score per model, broken down by the five benchmark categories. The total combines the 20 supported questions scored under Rubric A and the 10 unsupported questions scored under Rubric B, giving a maximum of 180 points per model.

Model	Total	Rubric A	Rubric B	Cat 1 (A | B)	Cat 2 (A | B)	Cat 3 (A | B)	Cat 4 (A | B)	Cat 5 (A | B)
	(/180)	**(/120)**	**(/60)**	**36 (24 | 12)**	**36 (24 | 12)**	**36 (24 | 12)**	**36 (24 | 12)**	**36 (24 | 12)**
`deepseek-r1:32b`	166	115	51	32 (23 | 9)	35 (24 | 11)	33 (23 | 10)	33 (23 | 10)	33 (22 | 11)
`gemma4:26b`	161	118	43	31 (24 | 7)	33 (23 | 10)	34 (23 | 11)	32 (24 | 8)	31 (24 | 7)
`gemma2:9b`	151	109	42	30 (22 | 8)	31 (20 | 11)	31 (21 | 10)	33 (24 | 9)	26 (22 | 4)
`gemma2:27b`	147	105	42	31 (23 | 8)	32 (22 | 10)	29 (20 | 9)	30 (20 | 10)	25 (20 | 5)

Table 5.5 Overall benchmark scores per model. Total is out of 180 (Rubric A out of 120 across the 20 supported questions, Rubric B out of 60 across the 10 unsupported questions). Each category column is structured as *total* (*A* | *B*), where the ceiling is 36 (24 | 12) — 36 category points split into 24 Rubric A points (4 supported questions at 6 each) and 12 Rubric B points (2 unsupported questions at 6 each). For example, `deepseek-r1:32b` on Category 1 scored 32 (23 | 9): 32 of the 36 available, split as 23 A-points and 9 B-points.

`deepseek-r1:32b` ranks first with 166 of 180 points, five ahead of `gemma4:26b` at 161. The

top two models reach the their totals through different profiles: `gemma4:26b` earns the highest Rubric A score in the benchmark (118 of 120) but loses ground on Rubric B (43 of 60), while `deepseek-r1:32b` trades a small amount of supported-question accuracy (115 of 120) for the strongest boundary behavior in the field (51 of 60). `gemma2:9b` and `gemma2:27b` trail by 15 and 19 points respectively. At the category level, three patterns stand out. First, Category 5 (built-in example models) is where the field separates most sharply: `deepseek-r1:32b` earns 33 of 36 while `gemma2:9b` and `gemma2:27b` earn 26 and 25, a 7–8 point spread that does not appear in any other category. Second, Category 3 (mathematical grammar and AST syntax) is `gemma4:26b`'s strongest category at 34 of 36, while the same category is `gemma2:27b`'s second-weakest at 29 — a five-point divergence between the two `gemma` instruction models. Third, Categories 1 and 2 are the most uniform, with all four models within four points of each other — consistent with the observation in Section 5.3.5 that procedural questions draw on well-covered chunks and tend to produce convergent answers.

5.3.2 Rubric Dimension Summary

The aggregate totals in Section 5.3.1 answer *how much* each model scored. Table 5.6 answers *on what*: it reports each model's score on every rubric dimension defined in Sections 5.2.1 and 5.2.2. The three S-dimensions are summed across the 20 supported questions (max 40 each); the three U-dimensions across the 10 unsupported questions (max 20 each). Each cell reports the earned and available points, followed by the percentage in parentheses. The three summary rows fold the six dimensions back into Rubric A, Rubric B, and the overall total, which reconcile with the columns reported in Table 5.5.

Dimension	gemma4	ds-r1	gemma2:27b	gemma2:9b
S1: Factual Grounding	40/40 (100.0%)	36/40 (90.0%)	38/40 (95.0%)	39/40 (97.5%)
S2: Completeness	38/40 (95.0%)	40/40 (100.0%)	29/40 (72.5%)	33/40 (82.5%)
S3: UI Actionability & Syntax	40/40 (100.0%)	39/40 (97.5%)	38/40 (95.0%)	37/40 (92.5%)
Rubric A (S1+S2+S3)	**118/120 (98.3%)**	**115/120 (95.8%)**	**105/120 (87.5%)**	**109/120 (90.8%)**
U1: Refusal Adherence	17/20 (85.0%)	20/20 (100.0%)	18/20 (90.0%)	18/20 (90.0%)
U2: Scope Justification	15/20 (75.0%)	18/20 (90.0%)	15/20 (75.0%)	13/20 (65.0%)
U3: Constructive Alternative	11/20 (55.0%)	13/20 (65.0%)	9/20 (45.0%)	11/20 (55.0%)
Rubric B (U1+U2+U3)	**43/60 (71.7%)**	**51/60 (85.0%)**	**42/60 (70.0%)**	**42/60 (70.0%)**
Total (A+B)	**161/180 (89.4%)**	**166/180 (92.2%)**	**147/180 (81.7%)**	**151/180 (83.9%)**

Table 5.6 Rubric dimension summary per model. Each cell is *earned* / *available* (*percentage*) summed across all questions to which the dimension applies. The three summary rows aggregate the S-dimensions into Rubric A, the U-dimensions into Rubric B, and both into the overall total.

Three patterns emerge. First, the S-dimensions are nearly saturated: every model clears 90% on S1, and no model falls below 92% on S3. The only S-dimension with a substantive spread is S2 (completeness), where the gap between the best and worst model is 27.5 percentage points (`deepseek-r1:32b` at 100% against `gemma2:27b` at 72.5%). Second, the U-dimensions expose the second widest divergence in the benchmark: U3 ranges from 45% (`gemma2:27b`) to 65% (`deepseek-r1:32b`), a twenty-point spread, and U2 ranges from 65% (`gemma2:9b`) to 90% (`deepseek-r1:32b`). Refusing is easy; being useful while refusing is not. Third, no model leads on every dimension. `gemma4:26b` is the only model with perfect S1 and S3 scores, but it is the weakest on U1 (85%) and offers a constructive alternative in only 55% of unsupported cases. `deepseek-r1:32b` leads on S2, U1, and U3, but trails `gemma4:26b` on S1 and S3. `gemma2:9b` trails `deepseek-r1:32b` on U1 by ten points and loses further ground on U2 and U3. `gemma2:27b` is the only model to fall below 50% on any dimension (U3 at 45%), the single largest deficit in the profile.

5.3.3 Boundary Adherence

Rubric B decomposes boundary behavior into three independent dimensions: whether the model refused the unsupported request rather than inventing a workflow (U1), whether it explained the architectural reason for the refusal (U2), and whether it suggested a valid SIRcraft workaround (U3). Each dimension is scored 0–2 per question, giving a maximum of 20 across the ten unsupported questions. Table 5.7 reports all three per model, plus the total Rubric B score.

Dimension (max 20)	`gemma4`	`ds-r1`	`gemma2:9b`	`gemma2:27b`
U1: Refused the request	17	**20**	18	18
U2: Named the boundary	15	**18**	13	15
U3: Offered a workaround	11	**13**	11	9
Rubric B total (max 60)	43	**51**	42	42

Table 5.7 Boundary adherence per model, decomposed by Rubric B dimension. Each of the top three rows scores 0–2 per unsupported question, summed across the ten questions for a maximum of 20. The final row reports the total Rubric B score.

Three findings emerge from the decomposition.

First, the binary refusal metric is the least discriminating of the three. All four models score 17 or higher on U1, and `deepseek-r1:32b` scores the maximum of 20. Refusal behavior is essentially solved; what varies is what the model does after refusing.

Second, `deepseek-r1:32b` is the only model that leads on every dimension. It refuses all ten requests, reaches the maximum U2 score on 8 of them, and reaches the maximum U3 score on 3. The gap to the second-place model is 8 points on Rubric B, and it comes from consistently higher performance across all three dimensions rather than from a single strength.

Third, the two `gemma2` models tie on Rubric B (42 each) but fail in opposite ways. `gemma2:9b` refuses correctly but reaches the maximum U2 score on only 4 of the 10 unsupported questions, usually because it stops at "this feature was not found in the documentation." `gemma2:27b` names the boundary more often (U2 = 15) but reaches the maximum U3 score on none of the ten questions — the weakest U3 in the field. The two failure modes are not equivalent for a user: a refusal that explains the boundary teaches the user something, while a refusal that offers a workaround keeps the user moving.

The only fabricated workflow in the benchmark appears in the U1 row. On Q5.6, `gemma2:9b` and `gemma2:27b` both invent a menu path that does not exist: *File* → *Examples* → *"Multi-City Airline Network Model"*. No such entry appears in SIRcraft's ten built-in examples, and a user following the instructions would find nothing. `gemma4:26b` and `deepseek-r1:32b` received the same seven retrieved chunks and both refused. This is the only question where a wrong answer takes the form of a fabricated operational workflow rather than an incomplete or ungrounded explanation, and the failure is a model property, not a retrieval limitation.

5.3.4 Syntax Validation Rate

The AST Syntax Validation Pass Rate measures how many of the equations a model produced would parse in SIRcraft without manual correction. Each equation in a model's `equations` array was extracted from the benchmark responses and passed unchanged to SIRcraft's own parser and function registry—the `Expression.g4` ANTLR grammar and the `SympyAstVisitor` that SIRcraft's simulator uses (Section 5.1.2). Table 5.8 reports the resulting pass rate per model, together with the failure-mode categories assigned by the validator.

Model	Pass Rate	Invalid	Syntax	Registry
`gemma4:26b`	100.0%	0 / 28	0	0
`gemma2:9b`	90.0%	2 / 20	2	0
`deepseek-r1:32b`	76.9%	9 / 39	3	6
`gemma2:27b`	63.6%	8 / 22	8	0
Field total	**82.6%**	**19 / 109**	**13**	**6**

Table 5.8 AST Syntax Validation Pass Rate per model, computed on the raw equations from each model's `equations` array. Columns report the model's pass rate, the invalid count over the total, and the split of those invalid entries between ANTLR syntax errors and function-registry errors.

No normalisation is applied: every entry in the `equations` array is tested as written, including extraction artifacts the pipeline captured from grammar demonstrations and prose listings, and including brace-wrapped duplicates that appear alongside their unwrapped counterparts. Two of the invalid entries in `deepseek-r1:32b`'s row are of this kind (Q5.3), and one (Q3.2) is a table row the model wrote in prose; none of the three is a genuine equation the model intended a user to paste.

The failures cluster into two categories, and each category belongs to a different subset of models. The *function-registry error* appears only in `deepseek-r1:32b` and covers two distinct model behaviors: four entries use `sum(...)`, a function SIRcraft does not define, on Q4.4; two use `max` and `min` on Q3.2, which the system prompt lists as supported but SIRcraft's registry does not implement (Section 5.4.3). The *ANTLR syntax error* appears in all three non-`gemma4` models: `gemma2:9b` and `gemma2:27b` both emit assignment-prefixed forms such as `b_2 = {0 : t <= 10, 0.02}` on Q5.4, and `gemma2:27b` additionally produces a malformed piecewise form (`{0 : t <= t_vacc, 0.02 : t > t_vacc}`) that omits the fallback clause SIRcraft requires. No failure mode is shared between the two `gemma` models and `deepseek-r1:32b`.

The pass rate treats assignment-prefixed forms as invalid because SIRcraft's evaluator cannot use them: the parser stops after the identifier and leaves the assignment unconsumed, so the

equation never runs as written. SIRcraft's own entry-time parser accepts such input silently and only reports a problem at simulation time, when the prefix names the same variable as the target field. The rate therefore reflects the user-facing failure mode rather than raw parse acceptance.

One distinction matters for RQ3 and would be invisible from the pass rate alone. `gemma2:27b`'s Q5.4 failures are unrecoverable for a user who does not already know the correct piecewise form, because no valid alternative appears anywhere in the answer. `gemma2:9b`'s Q5.4 failures are recoverable, because the same answer also contains the valid de-prefixed form. `deepseek-r1:32b`'s Q4.4 failures are informative rather than harmful: the four formulas are correct in content, the failure is only in the function name, and the guardrail warning names the exact problem. The post-processing gateway catches every syntax error before display, but it does not prevent them; the rate at which they are produced is a property of the model, not of the pipeline.

5.3.5 Retrieval Observations

The AST pass rate and the rubric scores measure the assistant's output, not the retrieval stage that feeds it. This subsection records what the retrieval pipeline did across the 30 questions, using the source lists attached to each response. The observations support RQ1 by showing where retrieval worked as designed, where the corpus itself introduced errors, and where post-processing added noise.

Retrieval was deterministic and topically aligned.

Every response across the 30 questions carries the same seven-chunk `SOURCES` list across all four models, confirming that the retrieval stage runs before model selection and does not depend on which model is chosen. A two-per-category sample of first-listed source chunks (one supported question and one unsupported question per category) illustrates the pattern: Chunk 11.9 for Q1.1, Chunk 10.13 for Q1.6; Chunk 11.1 for Q2.1, Chunk 17.3 for Q2.6; Chunk 9.9 for Q3.1, Chunk 9.9 for Q3.6; Chunk 5.14 for Q4.1, Chunk 19.1 for Q4.6; Chunk 20.1 for Q5.1, and Chunk 9.14 for Q5.6. For every supported question, the retrieved set contains at least one chunk whose content directly answers the question; the specific source chunk is not always the first-listed entry, but no supported question was answered from a set that lacked a topically matching chunk. For every unsupported question, the retrieved set sits in the closest documented area, because no chunk in the corpus answers the request directly. The full question list is given in Appendix B.

One corpus inconsistency propagated through the pipeline.

Chunk 9.22 (the tutorial for vector-borne modeling) writes the human infection rate as `bite * S_h * (I_v / N_v)`, using the total vector population in the denominator. Chunk 14.4 (the Sanusi et al. dengue specification) writes the same transition as `beta_hb_ * (I_v / N_h) * S_h`, using the total human population. The two chunks disagree both on the transmission parameter (`bite` against `beta_hb_`) and on the denominator. Both conventions are internally consistent within their chunk, and both chunks were retrieved on Q5.3. Three of the four models (`gemma2:9b`, `gemma2:27b`, `deepseek-r1:32b`) followed the tutorial denominator and produced `N_v` in the human infection equation; only `gemma4:26b` used the Sanusi denominator `N_h`. The failure is a knowledge-base defect, not a model error: the corpus contains two contradictory conventions for the same quantity, and the models had no basis for choosing between them.

Post-processing added two classes of noise.

The first is a backend heuristic that appends a fixed domain tag to any answer whose text contains *mosquito*, *malaria*, or *dengue* without the literal string "vector". The tag dates from early testing, before the retrieval pipeline was in place, and was kept as a lightweight cue for users who might not recognize a vector-borne model by name. With retrieval now supplying the actual source chunks, the tag is redundant on the answers it was meant for and misplaced on the answers it was not. It fires three times in the benchmark: `gemma2:9b` on Q2.3, where it duplicates what the answer already says; `gemma4:26b` on Q3.2 and Q5.6, where the answer merely mentions dengue as an example and the tag reads as out of context. It does not affect rubric scores on any of the three, but the tag is content that no retrieved chunk supports. Removing it is a one-line change to the RAG service and eliminates the class of noise entirely.

The second is the equation extraction pipeline, which captured illustrative function names (`sin`, `cos`, `sqrt`, and similar), brace-wrapped duplicates, and prose listings such as initial-size tables as if they were equations. These artifacts are included in the raw counts of Section 5.3.4 by design, following the counting method stated in Section 5.1.2. Both are properties of the pipeline, not of the models.

One model hedged against its own retrieved context.

On Q5.2, `deepseek-r1:32b` opened with "I couldn't find specific information about the Carcione et al. COVID-19 model in the provided context," then proceeded to provide the correct compartment values. Chunk 12.2, which carries those values, appears in the retrieved set for the same question, so the disclaimer is contradicted by the answer that follows it. The hedge concerns the attribution to Carcione et al. rather than the compartment data: the other three models named the model confidently, and only `deepseek-r1:32b` disclaimed that specific refer-

ence before answering. Either way, it is a presentation defect that the other three models did not exhibit.

The Salvager fired twelve times across the evaluation.

The Salvager runs when the model's JSON response cannot be parsed by either the direct decoder or the brace-extraction scan (Section 3.4.1), and recovers only the `answer` field. Fires are distributed unevenly: `gemma4:26b` once (Q2.1), `gemma2:9b` once (Q3.3), `deepseek-r1:32b` four times (Q3.2, Q3.5, Q4.3, Q5.4), and `gemma2:27b` six times (Q1.4, Q2.1, Q3.2, Q5.1, Q5.2, Q5.4). The distribution tracks each model's structured-output contract compliance rather than any particular question type; `gemma2:27b` is the weakest at producing parseable JSON, and `gemma4:26b` is the strongest. Salvager events do not affect rubric scores, but they do degrade the user experience: the answer's follow-up suggestions are replaced with the fallback defaults rather than model-generated follow-ups.

5.4 Discussion

5.4.1 RQ1: Knowledge Base Engineering and Vector Retrieval

RQ1 asks whether manual curation and semantic boundary chunking can produce a vector index that supports precise, domain-grounded retrieval for SIRcraft. The observations in Section 5.3.5 answer the retrieval half positively: the same seven-chunk set was retrieved across all four models for every question, every supported question drew at least one topically matching chunk, and no supported question was answered from a set that lacked supporting content. The manual chunking strategy held up because it preserved the boundaries that mattered: each chunk is topically self-contained, and no chunk was split in a way that separated an equation from its parameters or a piecewise rule from its fallback expression.

A second test of RQ1 is whether the retrieved set is narrow enough to avoid pulling in material that competes with the correct chunk. On Q5.3, the top-K set for the Sanusi dengue model included both Chunk 9.22 and Chunk 14.4, which write the human infection transition with different denominators. Three of the four models produced a hybrid that combines the parameter name of one chunk with the denominator of the other; only `gemma4:26b` produced the transition that matches the Sanusi source. Retrieval had no way to prefer one chunk, because both are topically relevant and both are written as authoritative. The underlying problem is that the two chunks were written from different sources and never reconciled, so the knowledge base contains two contradictory conventions for the same quantity.

A metadata filter on the retrieved chunks keyed to the question's topic class would narrow

the top-K set. On a question about a specific model like the Sanusi dengue specification, the filter would keep chunks tagged for that model and drop chunks tagged for general vector-borne modelling, so the model would see one convention instead of two. This is the smallest change that would reduce exposure to the Q5.3 class of failure, and it would also help on any question where the retrieved set contains more material than the question needs.

Taken together, the answer to RQ1 is positive but conditional. Manual curation and semantic boundary chunking do support domain-grounded retrieval: every supported question drew at least one topically matching chunk, and the retrieved set was identical across models. They do not on their own guarantee that the retrieved set is internally consistent or that it contains only what the question needs. The metadata filter above is the smallest change that would close part of that gap. Further work on making the corpus itself consistent is examined in Chapter 6.

5.4.2 RQ2: RAG Grounding and Operational Guidance

RQ2 asks whether retrieval-augmented generation can keep a language model grounded on SIRcraft documentation while producing instructions a user can act on. The evidence is positive on the first half and positive with one qualification on the second.

Factual accuracy was high across the field. `gemma4:26b` earned 118 of 120 Rubric A points, the highest in the benchmark and the only score above 115; `deepseek-r1:32b` followed at 115, and the two `gemma2` models at 109 and 105. No model invented a GUI feature on a supported question, and no supported answer sent the user to a menu path that does not exist. The retrieved chunks supplied the specific labels, parameter names, and syntax the models would otherwise have had to recall from their own training.

The questions that separated the models were those whose premise was wrong. On Q2.3, the question asked how to configure a Recovered compartment for mosquitoes; no such compartment exists in a vector model. `gemma4:26b`, `deepseek-r1:32b`, and `gemma2:9b` rejected the premise and named the actual vector structure (`S_v` → `I_v`); `gemma2:27b` rejected the premise but did not name the structure, and lost one S2 point for the omission. Q5.5 tested the same behaviour with an in-host viral template that does not exist; three models named the missing feature and refused, and `gemma4:26b` refused without naming it. Naming the requested feature is therefore the behaviour that separates the models on this class of question, not the refusal itself.

The category where the field separated most was completeness (S2). `deepseek-r1:32b` earned the maximum of 40; `gemma4:26b` lost two points, `gemma2:9b` lost seven, and `gemma2:27b` lost eleven. The losses came from omitting a secondary parameter, a format option, or a step that the source chunk listed, not from getting facts wrong. The pattern is therefore not a grounding failure but an exhaustiveness one: the weaker models found the right chunks and used the right facts, but stopped short of writing out everything the chunks supported.

Two anomalies are worth noting. On Q5.2, `deepseek-r1:32b` opened by saying it could not find information about the Carcione et al. model, then gave the correct compartment values that the retrieved chunk contained. The hedge concerns the attribution rather than the data; the answer that follows is accurate. On Q5.3, the corpus inconsistency described in Section 5.3.5 caused three of the four models to produce a human infection rate with the wrong denominator. The failure originates in the knowledge base, not in the models, and it is the clearest example of how RQ1 and RQ2 interact: a grounded answer can still be wrong when the sources themselves disagree.

The answer to RQ2 is therefore positive, with one qualification. Grounding works: the assistant produces answers that follow the retrieved documentation, and no supported question drew a fabricated GUI feature. The qualification is that grounding is only as good as the corpus. A user who trusted the assistant on Q5.3 would build a model with the wrong denominator, and the assistant would not warn them, because the retrieved chunks did not agree with each other in the first place.

5.4.3 RQ3: Post-Retrieval AST Validation

RQ3 asks whether the post-processing gateway can enforce SIRcraft's equation grammar before an equation reaches the user. The classifier was SIRcraft's own grammar and function registry (Section 5.1.2), so every invalid entry in Table 5.8 is an equation SIRcraft itself rejects. `gemma4:26b` produced no invalid equations, `gemma2:9b` produced two, `gemma2:27b` eight, and `deepseek-r1:32b` nine. The answer to RQ3 is positive on classification and negative on prevention: the gateway flags every invalid equation correctly, but it cannot stop the models from producing them.

The nine `deepseek-r1:32b` failures split into six function-registry errors and three syntax errors; the eight `gemma2:27b` failures are all syntax errors. Registry errors cover two behaviours. Four Q4.4 formulas call `sum(...)`, which SIRcraft does not implement; these are the informative case, because the formulas are mathematically correct and the failure is only in the function name. Two Q3.2 entries use `max` and `min`, which the system prompt lists as supported and SIRcraft's registry does not implement; this is a pipeline defect, not a model defect. The eight `gemma2:27b` syntax errors consist of assignment-prefixed forms such as `b_2 = {0 :  t <= 10, 0.02}` and malformed piecewise forms that omit the fallback clause SIRcraft requires; the assignment-prefix class is produced by both `gemma2` models. The three deepseek syntax errors are extraction artifacts — a prose table row captured as an equation on Q3.2, and two initial-size listings on Q5.3 — rather than model-emitted equation syntax. The assignment-prefix failures are a genuine model defect with a non-obvious user-facing consequence. SIRcraft's entry-time parser accepts the `=` prefix silently: it parses the identifier, leaves the assignment unconsumed, and reports no error until the next simulation run, when the model either raises a circular reference or runs

with a value other than the one intended. Rejecting prefixed forms at display time surfaces the problem before the user pastes anything into SIRcraft.

Q5.4 is the clearest example of why that matters. `gemma2:27b` answered with six equations, all six invalid, and no valid alternative anywhere in the response. `gemma2:9b` answered with four equations, two invalid and two valid; the invalid ones are the same expressions as the valid ones with the prefixes still attached. A user can recover the correct form from `gemma2:9b`'s answer by dropping the prefix, but cannot recover it from `gemma2:27b`'s. Same question, same retrieved chunks, different recoverability — visible only because the gateway reported the failures instead of silently dropping them.

The gateway is therefore validated on the classification task. Its limitation is that it operates after generation; preventing the models from producing invalid syntax in the first place would require decoding-time grammar enforcement, which the current implementation does not include.

5.4.4 RQ4: Negative Boundary Precision

RQ4 asks whether the assistant recognises unsupported requests and declines them without fabricating a workflow. The answer is positive on the primary axis and qualified on the two that follow it. All four models score at least 17 of 20 on U1, and `deepseek-r1:32b` scores the maximum. Refusing is effectively solved; what separates the models is what they do after refusing.

`deepseek-r1:32b` is the only model that leads on all three dimensions. It refuses all ten unsupported questions, reaches the maximum U2 score on eight of them, and reaches the maximum U3 score on three. The gap to the second-place model is eight points on Rubric B, and it comes from higher performance across all three dimensions rather than from a single strength. The three remaining models each have a specific weakness. `gemma4:26b` refuses every unsupported question but reaches the maximum U1 score on only seven; on the other three (Q1.6, Q4.6, Q5.5) it falls back to the fixed refusal template without naming the requested feature, and a user reading a template refusal does not learn what was out of scope. `gemma2:9b` refuses correctly but reaches the maximum U2 score on only four of the ten, because its refusals usually stop at "this feature was not found in the documentation." `gemma2:27b` names the boundary more often (U2 = 15) but reaches the maximum U3 score on none of the ten — the weakest U3 in the field, and the only dimension on which any model scores zero on the maximum-score count.

The only fabricated workflow in the benchmark appears in the U1 row. On Q5.6, `gemma2:9b` and `gemma2:27b` both invent a menu path that does not exist: *File → Examples → "Multi-City Airline Network Model"*. No such entry appears in SIRcraft's ten built-in examples, and a user following the instructions would find nothing. `gemma4:26b` and `deepseek-r1:32b` received the same seven retrieved chunks and both refused. This is the only question on which the assistant actively

misleads rather than declines, and the failure is a model property, not a retrieval limitation.

Two patterns cut across the categories. Whole-paradigm questions — stochastic processes, streaming data, Monte Carlo sweeps, GIS mapping, agent-based models — produce a dead-end refusal (U1=2, U2=2, U3=1) in most models that encounter them: the model recognises the boundary, explains it, and then stops. Questions with a nearby valid alternative (Q3.5, the LaTeX syntax trap, and Q5.6, the airline network model) score higher, because the model can offer a substitute rather than only a refusal. The second pattern is that informative refusals are rare in absolute terms: of the forty U1 scores across all four models and all ten unsupported questions, only 35 reach the maximum of 2, and no model reaches the maximum U3 score on more than three of the ten.

RQ4 is therefore answered positively, with two qualifications. The assistant reliably refuses unsupported requests: no model hallucinates on more than one unsupported question, and every model reaches at least 90% on the binary boundary metric. But refusal quality is not uniform. The fixed refusal template in the system prompt limits how often a refusal is informative, and no model offers a fully informative refusal with a concrete workaround in more than three of the ten unsupported cases. The Q5.6 hallucination shows that the boundary is not unconditionally reliable when the question names a specific missing feature.

Chapter 6

Conclusion and Future Work

6.1 Conclusion

6.1.1 Problem Restatement

This thesis addressed a gap between the accessibility of SIRcraft's modelling interface and the conceptual expertise required to use it. The platform itself solves the software barrier: it is free, license-free, offline-capable, and provides a no-code graphical editor for arbitrary deterministic compartmental models. The barrier it does not solve is the modelling barrier. Translating a biological transmission mechanism into a valid rate equation, configuring piecewise interventions, and navigating multi-step GUI workflows all require domain knowledge that the platform's target users do not necessarily have.

General-purpose language models can assist with parts of this task, but they present three problems for this setting. They are not grounded in SIRcraft's documentation, so they can invent features that do not exist. They default to LaTeX markup and other formatting that SIRcraft's equation parser rejects. And they require sending queries and institutional research material to external commercial services, which conflicts with the data-governance requirements of a public university.

The work in this thesis built and evaluated a domain-grounded RAG assistant that addresses these three problems within the university network boundary. A hand-curated SIRcraft knowledge base and a local retrieval pipeline keep the answers grounded in the platform's own documentation. A canonical prompt, shared unchanged across the evaluated models, enforces the behavioural rules and the equation syntax the platform accepts. A post-processing gateway validates every generated equation against SIRcraft's grammar before the answer reaches the user.

6.1.2 Main Research Question

The main research question asked how effectively a local, domain-grounded RAG system could assist users in constructing epidemiological compartmental models, guiding GUI operations, and enforcing negative system boundaries within SIRcraft.

The evaluation in Chapter 5 answers this in the affirmative on all three axes.

Grounding. No model fabricated a GUI feature on a supported question, and every supported question drew at least one topically matching chunk from the retrieved set.

Boundary enforcement. Every model refused at least nine of the ten unsupported questions, and no model fabricated a workflow on more than one unsupported question. On one supported question (Q5.3), three of the four models produced a human infection rate that matched one retrieved chunk but not the Sanusi source chunk; the underlying cause was a knowledge-base inconsistency between two conflicting chunks, not a model fabrication.

Syntax validation. Every invalid equation was flagged before display, and every flag named the specific parse error.

The qualifications concern quality rather than capability. Retrieval is consistent in coverage but not in internal consistency: on one question the retrieved set contained two chunks that disagreed on the same transition, and three of the four models produced an equation that matched neither source exactly. Boundary refusals are reliable in principle but uneven in quality: a model that refuses without naming the requested feature leaves the user less informed than one that names the boundary and offers a workaround. Post-processing catches every grammar violation, but it cannot prevent the models from producing them in the first place. Each of these is examined in Section 6.2.

6.1.3 Sub-Question Answers

RQ1 — Knowledge Base Engineering and Vector Retrieval.

The knowledge base design supports precise retrieval for SIRcraft. Manual curation and semantic boundary chunking preserve the structural coherence of the source documents, and every supported question in the benchmark drew at least one topically matching chunk from the retrieved set. Two design limitations were identified. First, manual curation does not guarantee consistency between chunks that describe the same concept from different sources. On one question the retrieved set contained two chunks that disagreed on the same transition, and three of the four models produced an equation that matched neither source exactly. Second, the retrieved set is broad rather than narrow: the returned chunks are all topically relevant, but they can include adjacent material that competes with the correct chunk on questions that have a specific answer.

Both limitations have concrete mitigations: a metadata filter keyed to the question's topic class (Section 5.4.1), and a corpus consistency pass before ingestion (Section 6.2)

RQ2 — RAG Grounding and Operational Guidance.

Retrieval-augmented generation produced grounded operational guidance on all supported questions in the benchmark. No model fabricated a button, a menu path, or a parameter that does not exist in SIRcraft, and the strongest model in the field earned 118 of the 120 Rubric A points available. TThe differences between models on this axis are about completeness rather than about accuracy: the weaker models omitted secondary parameters, format options, or configuration steps that the source chunks listed, but did not fabricate GUI features or menu paths. On one supported question (Q5.3), three of the four models produced a human infection rate that mixed the Sanusi parameter name with a denominator drawn from a different retrieved chunk; the underlying cause was a knowledge-base inconsistency between two conflicting chunks, not a model fabrication Grounding is therefore a solved axis for the assistant as a whole, and the remaining work is to close the completeness gap between the strongest and weakest models.

RQ3 — Post-Retrieval AST Validation.

The post-processing gateway reliably detects SIRcraft grammar violations and surfaces them with a specific reason, and the false-rejection case is narrow: the validator's whitelist omits one function (log) that SIRcraft's evaluator does implement, and no equation using that function appeared in the benchmark. The reverse direction is not uniform. The gateway's function whitelist contains three names that SIRcraft's evaluator does not implement (`max`, `min`, and `if`), so an equation using any of them passes validation and would fail when pasted into SIRcraft. The benchmark exercised two of these names, `max` and `min`, on a single question. The gateway also catches grammar violations at a different stage than SIRcraft does: the assistant rejects invalid syntax at display time, while SIRcraft's own parser accepts most invalid forms at entry and only reports a problem at simulation. The two validators therefore disagree about timing and about failure mode, and their disagreements are confined to four function names: the validator accepts three functions (`max`, `min`, `if`) that SIRcraft does not implement, and rejects one function (`log`) that it does.

RQ4 — Negative Boundary Precision.

The assistant reliably declines unsupported requests, and the prompt-level boundary is largely enforced across models. Every model refused at least nine of the ten unsupported questions, and the field's boundary precision on the binary metric was high enough that the interesting variation is in refusal quality rather than in refusal presence. Two quality gaps were identified. The first is

that the fixed refusal template in the system prompt caps how often a model produces an informed refusal: a model that follows the template exactly cannot name the requested feature, even though doing so is what makes the refusal informative. The second is that concrete workarounds are rare: the best-performing model offered one on only three of the ten unsupported questions, and no other model exceeded that count. The two fabricated menu paths on Q5.6 (one each by `gemma2:9b` and `gemma2:27b`) show that the boundary is not unconditionally reliable, but the failure was confined to a single question.

6.1.4 Contributions

This thesis makes four contributions.

A domain-grounded RAG system for SIRcraft.

The thesis designed, built, and deployed a complete RAG assistant that operates entirely within the University of Bremen network. The system is available to users through the SIRcraft project website, runs on two dedicated university servers, and requires no external API keys or cloud services. The codebase, the curated knowledge base, and the deployment configuration are released with the thesis.

A curated knowledge base and a repeatable ingestion pipeline.

The knowledge base was manually gathered from four source categories (thesis documentation, built-in example specifications, video tutorial transcripts, and application manual entries) and processed through a normalization and semantic-chunking pipeline that preserves the structural coherence of the source documents. The pipeline is repeatable and can be re-run whenever the source material changes.

A two-rubric evaluation benchmark for domain-grounded assistants.

The benchmark decomposes assistant behaviour into three dimensions per question class. On the supported side the dimensions are factual grounding, response completeness, and UI actionability including syntax validity; on the unsupported side they are refusal rule adherence, scope justification, and constructive alternative. The two-rubric design separates two failure modes that a single score would average together: an assistant can be factually accurate but incomplete, or correctly refusing but uninformative, and the rubric reports the difference. The 30-question benchmark is included as Appendix B; the two rubrics are defined in Chapter 5.

An empirical comparison of four open-weights models on the benchmark.

Four locally hosted models were evaluated against the same system prompt, the same knowledge base, and the same retrieval pipeline. The comparison separates pipeline properties (retrieval coverage, corpus consistency) from model properties (completeness, refusal quality, equation syntax reliability).

6.2 Future Work

The evaluation identified a small number of improvements that would each address a concrete failure mode observed in Chapter 5.

Corpus consistency pass.

The single highest-impact change is a consistency check before ingestion. Two chunks in the current corpus describe the same quantity with different conventions, and the retrieval stage cannot distinguish between them at query time. A rule-based pass that detects conflicting definitions of the same symbol across chunks would eliminate the entire class of failure, and would need to run before every ingestion so that future additions inherit the same guarantee.

Administrative knowledge base UI.

The current ingestion pipeline is a command-line script that rebuilds the index from scratch. A small web interface for the corpus would lower the barrier to keeping it current and would let a curator detect and resolve three classes of issue directly: duplicate chunks that say the same thing, outdated chunks whose content no longer matches the SIRcraft application, and conflicting chunks that disagree with each other. The interface should surface the consistency-pass findings described above and offer inline edit and delete operations, with the re-index triggered automatically on save. An admin page of this kind is a small addition to the existing backend, and it would make the corpus maintainable by someone other than the thesis author.

An agent integrated into SIRcraft.

The assistant in its current form is a separate web application that a user visits alongside the SIRcraft desktop client. The natural extension of this work is an in-application agent: a panel inside SIRcraft that has direct access to the current model state, the user's canvas, and the equation editor. An in-application agent could do three things the current system cannot. It could answer questions relative to the model the user is currently building rather than to the

general knowledge base. It could propose edits directly in the canvas, with an explicit user confirmation step, rather than describing the steps in prose. And it could refuse to suggest equations that would fail to parse before the user pastes them, because it would call the same evaluator the simulation engine uses. The core components of the current assistant — the knowledge base, the retrieval pipeline, and the grammar validator — would transfer to this design without modification; only the presentation surface and the state access would be new.

Appendices

Appendix A

Frontend Dependencies

This appendix records the direct dependencies declared in `frontend/pubspec.yaml`, together with the version constraints pinned at the time of the evaluation reported in Chapter 5.

The application targets the Dart SDK constraint `^3.11.5` and is built against the stable Flutter channel. All packages are installed via `flutter pub get`.

Runtime dependencies

Package	Version	Purpose
`cupertino_icons`	`^1.0.8`	iOS-style icon set
`collection`	`^1.18.0`	List and map utilities
`http`	`^1.6.0`	HTTP client for backend calls
`flutter_markdown`	`^0.7.7+1`	Markdown rendering widget
`markdown`	`^7.3.1`	Markdown parser
`flutter_bloc`	`^9.1.0`	BLoC/Cubit state management
`equatable`	`^2.0.7`	Value equality for state classes
`url_launcher`	`^6.3.0`	Opening links found in answers
`dart_mappable`	`^4.3.0`	JSON serialisation annotations
`shared_preferences`	`^2.5.5`	Persisted theme choice

Direct runtime dependencies of the Flutter frontend.

Appendix B

The 30-Question Benchmark Dataset

The benchmark consists of 30 questions distributed evenly across five functional categories, with four supported and two unsupported questions per category (20 supported, 10 unsupported overall). Each question was submitted to the assistant in isolation, with no conversation history carried over and the model selector set before submission (Section 5.1.2). Unsupported questions are marked with an asterisk.

Category 1 — Platform Setup, Packaging, and UI Navigation

1.1 *(supported)* How do I lock a completed model in SIRcraft to prevent accidental edits, and which UI controls remain functional while locked?

1.2 *(supported)* Where do I navigate to export the numerical time-series data of my simulation, and what columns does the resulting CSV file include?

1.3 *(supported)* When exporting a complete model package from the File menu, what exact file types are contained within the generated ZIP archive?

1.4 *(supported)* What package formats are available for installing SIRcraft on Linux distributions such as Ubuntu or Debian?

1.5 *(unsupported)* How do I export my simulation visualization as an interactive 3D WebGL HTML bundle in SIRcraft?

1.6 *(unsupported)* Where in the settings do I configure AWS S3 or Google Drive API credentials to enable automated cloud backup syncing?

Category 2 — Compartmental Canvas Architecture and Multi-Species Modelling

2.1 *(supported)* How do I create a new compartment on the canvas, and what are its three configurable properties?

2.2 *(supported)* How do I connect two compartments with a transition arrow, and how do I configure move versus birth/death transitions?

2.3 *(supported, synthesis trap)* When modeling vector-borne Dengue fever with human and mosquito populations on the same canvas, how do I configure the Recovered compartment for mosquitoes?

2.4 *(supported)* What visual indicators does SIRcraft display on the canvas to represent hidden mathematical dependencies and unlinked cross-population references?

2.5 *(unsupported)* How do I assign GIS latitude and longitude coordinates to compartments to simulate geographic spatial spread across a district map?

2.6 *(unsupported)* How do I assign individual agent attributes (such as age, mask-wearing compliance, and daily contacts) to individual entities in the model?

Category 3 — Mathematical Grammar, AST Syntax, and Solvers

3.1 *(supported)* How do I write a standard mass-action infection transition rate `beta * S * I / N` using custom parameters?

3.2 *(supported)* What built-in mathematical functions does the equation parser support, and how do I formulate annual trigonometric seasonality for transmission rates?

3.3 *(supported)* How do I write a conditional piecewise equation in SIRcraft to reduce the contact rate by 50% after simulation day 30?

3.4 *(supported)* Can I enter direct fractional arithmetic expressions like `11 / 5` or `1 / 69` into constant variables, and how does the engine evaluate them?

3.5 *(unsupported, syntax trap)* Can I format my transition rate equations using standard LaTeX syntax such as `\frac{\beta \cdot S \cdot I}{N}`?

3.6 *(unsupported)* How do I embed a stochastic Brownian motion noise term `dW(t)` or Poisson jump process directly into a transition rate?

Category 4 — Dynamic Scenarios, Visualizations, and Validation

4.1 *(supported)* How do I add an interactive parameter slider to the visualizations dashboard, and how does the simulation update when the slider moves?

4.2 *(supported)* How do I duplicate an entire model branch on the canvas to evaluate a baseline scenario against an intervention scenario in the same plot?

4.3 *(supported)* How do I add dynamic reference lines on an epidemic curve that automatically track an intervention start parameter `T`?

4.4 *(supported)* What clinical error metrics does SIRcraft support for comparing simulations against imported empirical time-series data?

4.5 *(unsupported)* How do I configure a live REST API webhook in SIRcraft to continuously stream incoming daily hospitalization numbers into the running simulation?

4.6 *(unsupported)* Where is the menu option in SIRcraft to configure an automated 10,000-run Monte Carlo parameter sweep with Latin Hypercube sampling?

Category 5 — Built-in Benchmark Case Studies

5.1 *(supported)* In the built-in J. D. Murray influenza model, what are the configured simulation timesteps, time unit, and hourly resolution settings?

5.2 *(supported)* What compartments comprise the Carcione et al. COVID-19 model, and what initial population values are assigned to Lombardy Case 1?

5.3 *(supported)* How does the built-in Sanusi et al. Dengue model formulate cross-species transmission between human host compartments and mosquito vector compartments?

5.4 *(supported)* In the built-in Turkyilmazoglu SIRVI vaccination model, how are moderate vaccination and strong vaccination with waning immunity parameterized?

5.5 *(unsupported)* Which built-in template in SIRcraft models in-host intra-cellular viral dynamics between free virions and CD4+ T-cells?

5.6 *(unsupported)* How do I load the built-in multi-city airline transportation network model to simulate cross-border airport transmission hubs?

Appendix C

Canonical System Prompt

Listing C.1 The canonical system prompt, sent unchanged to every model in the evaluation set (part 1 of 2: persona, grammar reference, and behavioural rules).

```
You are the SIRcraft Assistant, an expert in epidemiological modeling and the SIRcraft software.
ur tone is professional, helpful, and technically precise.

Rcraft MATHEMATICAL GRAMMAR REFERENCE:
Numbers: Integers (762), Decimals (0.002), Scientific Notation (1.5e-3).
Arithmetic: +, -, *, /, ^ (exponentiation).
Comparison: ==, !=, <, <=, >, >=.
Functions: sin(x), cos(x), sqrt(x), exp(x), ln(x), abs(x), max(a,b), min(a,b).
PIECEWISE (ABSOLUTE REQUIREMENT): Piecewise logic MUST start with '{' and end with '}'. NO EXCEPTIONS. Example: { expr1 : cond1, fallback }. Use a COLON (:) for conditions and a COMMA (,) for branches.
References: You can reference any variable or compartment by name (e.g., S, I, beta).
DO NOT use 'if/else' statements or differential notation (dS/dt).
IMPORTANT: LaTeX syntax (e.g., \frac, \beta, \times) is NOT supported and MUST NOT be used.

  CRITICAL RULES:
  1. CONVERSATIONAL CONTINUITY: Always reference previous parts of the conversation if relevant. Do not treat each question as isolated.
  2. USE PROVIDED CONTEXT: Base your answer STRICTLY on the context provided and the conversation history. Do not use outside knowledge for SIRcraft-specific features.
  3. EXHAUSTIVE DETAIL FOR NEW INFO: Provide thorough and detailed answers for new topics. If a process (like opening a project) was already explained, briefly reference it instead of repeating the full steps.
  4. VISUAL STRUCTURE: Use Markdown headers (###), bold text, and bullet points. Use double newlines ONLY between paragraphs and major sections.
  5. NO INTERNAL METADATA: NEVER output internal markers such as '[PRIMARY CHUNK]'. DO NOT mention chunk numbers or source titles in your answer.
  6. NO HALLUCINATED LINKS: NEVER provide URLs that are not explicitly listed in the provided context.
  7. MATH SYNTAX: Use plain-text mathematical syntax suitable for SIRcraft.
     - BACKTICKS: Wrap inline variables and numbers in single backticks (e.g., `beta` or `0.0323`). Wrap standalone equations in TRIPLE backticks (e.g., ```beta * S * I / N```). CRITICAL: DO NOT put English labels or colons inside the backticks. (WRONG: `Rate (mu_h): 0.0323`, RIGHT: Rate (mu_h): `0.0323`).
     - PIECEWISE: Use the syntax '{ expr1 : cond1, fallback }'. MANDATORY: Always use curly braces for piecewise logic.
     - NO LATEX: Do not use LaTeX artifacts or Greek symbols (\beta, \lambda). Use *, /, and plain variable names.
     - NO LABELS IN SYNTAX: The 'syntax' field in your JSON must contain ONLY the raw mathematical expression (e.g., `beta * S` or `0.0323`). Do not include English descriptions, prefixes, or labels inside the syntax string.
     - Extract ALL actual, model-ready mathematical equations into the 'equations' array of your JSON response.
  8. UNKNOWN INFO: If the context doesn't contain the answer, say: 'I'm sorry, but I couldn't find information about that in the SIRcraft knowledge base.'
  9. SUGGESTIONS: Generate exactly 3 relevant follow-up questions.
  10. JSON FORMAT: Output ONE single JSON object. The 'answer' field MUST be the first key in the object. Do not wrap the JSON in other text unless you are a reasoning model using <think> tags.

  MANDATORY RULES:
  - Your final output MUST be a valid JSON object.
  - THE 'answer' KEY MUST BE FIRST.
  - If you are a reasoning model, you may use <thought> or <think> tags to reason first, but you MUST provide the complete JSON object after the closing tag.
  - CONCISENESS: Keep your internal reasoning brief. Do not spend more than 20% of your token budget on thinking.
  - DO NOT output multiple JSON blocks.
  - DO NOT omit the JSON block even if you have explained everything in your reasoning.
```

Listing C.2 The canonical system prompt, sent unchanged to every model in the evaluation set (part 2 of 2: few-shot examples).

```
FEW-SHOT EXAMPLES:

Example 1: SIR Rate Transition
User: 'How do I model the infection transition?'
Assistant: {
  "answer": "In SIRcraft, the infection transition represents individuals moving from the `Susceptible (S)` compartment to the `Infected (I)` compartment. You can define this rate using a transmission parameter, often called `beta`.\n\n### Equation Setup\n1. Create a variable named `beta` and set it to a small value like `0.002`.\n2. Drag a transition from `S -> I` and input the following equation: `beta * S * I`.",
  "equations": [{"name": "Infection Transition", "syntax": "beta * S * I"}],
  "suggestions": ["How to add a slider for beta?", "What is frequency-dependent transmission?", "How to set initial compartment sizes?"]
}

Example 2: Piecewise Intervention
User: 'How do I reduce transmission by 50% after day 30?'
Assistant: {
  "answer": "To model a timed intervention like a lockdown, use **Piecewise Logic** to modify your transmission rate based on the simulation time `t`.\n\n### Implementation\n1. Create a constant variable `T_start = 30`.\n2. Update your infection transition equation to use a piecewise function that checks if `t` has exceeded `T_start`: `{ beta * 0.5 : t > T_start, beta }`.",
  "equations": [{"name": "Timed Intervention", "syntax": "{ beta * 0.5 : t > T_start, beta }"}],
  "suggestions": ["How to model multiple intervention phases?", "Can I use a slider for the start day?", "How to visualize the intervention on the plot?"]
}

Example 3: Unsupported Feature
User: 'Can SIRcraft run stochastic simulations or Monte Carlo loops?'
Assistant: {
  "answer": "I'm sorry, but I couldn't find information about stochastic simulations or Monte Carlo loops in the SIRcraft knowledge base. Currently, SIRcraft focuses on deterministic compartmental models based on user-defined transition equations.",
  "equations": [],
  "suggestions": ["What modeling types are supported?", "How to run a standard simulation?", "How to export results?"]
}

EXAMPLE JSON RESPONSE:
{
  "answer": "To model a seasonal rate, use a piecewise function...",
  "equations": [
    {"name": "Seasonal Transmission", "syntax": "{ beta * 1.5 : t > 100, beta }"}
  ],
  "suggestions": ["How to add a slider?", "What is piecewise syntax?", "How to run simulation?"]
}
```

Appendix D

Use of AI-based applications

No.	AI-based tool	Purpose	Affected chapters	Description of input (prompt)	Remark
1	Gemini	Brainstorming	Overall concept	*"What AI architectures are suitable for a domain-specific assistant that must ground its answers in a fixed documentation corpus and run offline?"*	Pointed toward retrieval-augmented generation, ChromaDB, and local embedding models as possible directions. The final choice of architecture was made by the author.
2	Gemini	Stack research	Technology stack	*"How can FastAPI serve a Flutter web frontend and communicate with a local vector store and a local LLM runtime?"*	Suggested the FastAPI + Flutter + ChromaDB + Ollama combination as one option. The stack was ultimately chosen by the author based on the requirements in Chapter 3.
3	Gemini	Deployment research	Deployment config	*"How do I configure Nginx to serve a compiled Flutter web bundle and reverse-proxy API requests to a FastAPI container in Docker Compose?"*	Offered configuration suggestions. The settings were tested and adapted against the running deployment on the University of Bremen servers.
4	Gemini Agent	Code debugging	Backend, frontend	*"Why are equations from the model response not being extracted correctly by the equation service, and how do I fix the extraction pipeline?"*	Offered analysis of the extraction pipeline and suggested corrections. The fixes were reviewed and verified by re-running the project test suite.

No.	AI-based tool	Purpose	Affected chapters	Description of input (prompt)	Remark
5	Gemini Agent	Launch scripts	Dev scripts	*"Write a shell script that activates the Python environment, loads the environment variables, and starts the FastAPI server with Uvicorn."*	Suggested initial script drafts. The scripts were tested in the author's environment and adapted as needed.
6	Gemini Agent	Test scripts	Test suite	*"Help me write pytest cases for the retrieval service that mock the Ollama embedding endpoint."*	Offered test scaffolding. The cases were reviewed, completed, and executed by the author; the final suite passes.
7	Gemini Agent	Automation scripts	Ingestion scripts	*"Write a script that rebuilds the ChromaDB collection from the knowledge base text file and reports the final chunk count."*	Assisted with automation of ingestion and re-indexing. The scripts were run against the actual corpus and adjusted as needed.
8	DeepSeek	LaTeX tables and figures	Evaluation tables	*"Convert these results into a LaTeX table with a caption, label, and consistent formatting."*	Assisted with the layout and formatting of tables and figures. All values were supplied by the author; the tables were compiled and visually checked.

23.09.2026 Khaled Swaidan

Glossary

Abstract Syntax Tree (AST)
A tree representation of a parsed equation. SIRcraft builds one from the concrete syntax tree produced by `Expression.g4`; the assistant's validator builds an equivalent AST from its recursive-descent parser.

Benchmark dataset
The 30-question evaluation set used in this thesis: four supported and two unsupported questions in each of five functional categories, listed in Appendix B.

Chunk
A self-contained unit of knowledge-base text. The SIRcraft corpus contains 179 chunks, each corresponding to one feature, parameter, or workflow.

Compartment
A state in a compartmental epidemiological model (e.g., Susceptible, Infected, Recovered) that individuals occupy at any one time.

Embedding
A fixed-length dense vector representation of a text. The assistant uses `nomic-embed-text` to produce 768-dimensional embeddings of every chunk and every user query.

Equation extraction
The pipeline stage that isolates candidate equations from a model's JSON response and prose, normalises them, and passes them to the validator.

Expression.g4
SIRcraft's ANTLR 4 grammar file that defines the equation syntax the simulator accepts.

Function registry
The set of named mathematical functions SIRcraft's evaluator implements. The registry and the assistant's validator disagree on four names: `max`, `min`, `if`, and `log`.

Guardrail warning
A user-facing notice appended to an answer when an extracted equation fails the assistant's grammar validator. The warning names the specific error; the equation is preserved.

HNSW
Hierarchical Navigable Small World — the graph index ChromaDB uses for approximate nearest-neighbour search over the embedding space.

Knowledge base
The hand-curated corpus of 179 SIRcraft-related chunks that grounds every assistant answer.

Negative boundary
The set of feature requests SIRcraft does not support (e.g., stochastic models, GIS mapping, agent-based models). The assistant is designed to refuse such requests without fabricating a workflow.

Ollama
The local inference runtime that serves both the `nomic-embed-text` embedding model and the four evaluated language models on the AI GPU host.

Piecewise expression
A conditional equation of the form `{ expr1 : cond1, fallback }`, used in SIRcraft to model timed interventions and other conditional behaviour.

Retrieval
The stage that embeds a user question and returns the top-K ($K = 7$)most similar chunks from the knowledge base.

Retrieval-augmented generation (RAG)
A generation architecture that conditions a language model's output on passages retrieved from an external index at query time.

Rubric A
The three-dimension rubric applied to supported questions: factual grounding (S1), response completeness (S2), and UI actionability including syntax validity (S3), each scored 0–2.

Rubric B
The three-dimension rubric applied to unsupported questions: refusal rule adherence (U1), scope justification (U2), and constructive alternative (U3), each scored 0–2.

Salvager
The third rung of the assistant's JSON recovery ladder. When a model's response cannot be parsed as JSON, the Salvager extracts only the `answer` field and discards the structured metadata.

SIR model
The Susceptible–Infected–Recovered compartmental model introduced by Kermack and McKendrick [KM27]; the canonical structure that SIRcraft supports and extends.

SIRcraft
The no-code, offline-capable desktop application for building arbitrary deterministic compartmental models, developed by Grebe [Gre25].

Transition
A directed flow between compartments in a SIRcraft model (e.g., Susceptible to Infected). Each transition carries a rate equation.

Vector store
The persistent index of chunk embeddings. The assistant uses ChromaDB, an in-process vector database with an HNSW index.

Bibliography

[BGB+13] Samir Bhatt, Peter W. Gething, Oliver J. Brady, Jane P. Messina, Andrew W. Farlow, Catherine L. Moyes, John M. Drake, John S. Brownstein, Anne G. Hoen, Osman Sankoh, Monica F. Myers, Dylan B. George, Thomas Jaenisch, G. R. William Wint, Cameron P. Simmons, Thomas W. Scott, Jeremy J. Farrar, and Simon I. Hay. The global distribution and burden of dengue. *Nature*, 496(7446):504–507, 2013.

[CHOL10] Dennis L. Chao, M. Elizabeth Halloran, Valerie J. Obenchain, and Ira M. Longini, Jr. FluTE, a publicly available stochastic influenza epidemic simulation model. *PLOS Computational Biology*, 6(1):e1000656, 2010.

[Chr24] ChromaDB Contributors. ChromaDB: The AI-native open-source embedding database, 2024. Version 0.6.3. Available at *https://www.trychroma.com/*. Accessed 2026-09-15.

[Dee25] DeepSeek-AI. DeepSeek-R1: Incentivizing reasoning capability in LLMs via reinforcement learning. *arXiv preprint arXiv:2501.12948*, 2025.

[ESDB07] Martin Eichner, Markus Schwehm, Hans-Peter Duerr, and Stefan O. Brockmann. The influenza pandemic preparedness planning tool InfluSim. *BMC Infectious Diseases*, 7(17):1–14, 2007.

[FR14] Scott Fortmann-Roe. Insight Maker: A general-purpose tool for web-based modeling & simulation. *Simulation Modelling Practice and Theory*, 47:28–45, 2014.

[Gem24] Gemma Team. Gemma 2: Improving open language models at a practical size. *arXiv preprint arXiv:2408.00118*, 2024.

[Gem26] Gemma Team. Gemma 4 technical report. Technical report, Google DeepMind, June 2026.

[Gre25] Aljoscha Grebe. A no-code approach to modelling and simulation of infectious diseases. Master's thesis, University of Bremen, Bremen, Germany, 2025.

[HG19] Peter Heidrich and Thomas Götz. Modelling dengue with the SIR model. In István Faragó, Ferenc Izsák, and Péter L. Simon, editors, *Progress in Industrial Mathematics at ECMI 2018*, volume 30 of *Mathematics in Industry*, pages 175–182, Cham, 2019. Springer International Publishing.

[HSH+25] Xuhan Huang, Qingning Shen, Yan Hu, Anningzhe Gao, and Benyou Wang. LLMs for mathematical modeling: Towards bridging the gap between natural and mathematical languages. In *Findings of the Association for Computational Linguistics: NAACL 2025*, 2025.

[KM27] William Ogilvy Kermack and Anderson Gray McKendrick. A contribution to the mathematical theory of epidemics. *Proceedings of the Royal Society of London. Series A*, 115(772):700–721, 1927.

[LPP+20] Patrick Lewis, Ethan Perez, Aleksandra Piktus, Fabio Petroni, Vladimir Karpukhin, Naman Goyal, Heinrich Küttler, Mike Lewis, Wen-tau Yih, Tim Rocktäschel, Sebastian Riedel, and Douwe Kiela. Retrieval-augmented generation for knowledge-intensive NLP tasks. In *Advances in Neural Information Processing Systems (NeurIPS)*, volume 33, pages 9459–9474, 2020.

[Mor20] Sinead E. Morris. *shinySIR: Interactive Plotting for Mathematical Models of Infectious Disease Spread*, 2020. R package version 0.1.2.

[MY20] Yury A. Malkov and Dmitry A. Yashunin. Efficient and robust approximate nearest neighbor search using hierarchical navigable small world graphs. *IEEE Transactions on Pattern Analysis and Machine Intelligence*, 42(4):824–836, 2020.

[NMDM25] Zach Nussbaum, John X Morris, Brandon Duderstadt, and Andriy Mulyar. Nomic embed: Training a reproducible long context text embedder. *Transactions on Machine Learning Research*, 2025.

[Oll24] Ollama Team. Ollama: Get up and running with large language models locally, 2024. Version 0.32.6. Available at *https://ollama.com/*. Accessed 2026-09-15.

[Pan24] Pan American Health Organization. Report on the epidemiological situation of dengue in the americas. Technical report, PAHO/WHO, November 2024.

[Par13] Terence Parr. *The Definitive ANTLR 4 Reference*. Pragmatic Bookshelf, 2013.

[RG19] Nils Reimers and Iryna Gurevych. Sentence-BERT: Sentence embeddings using siamese BERT-networks. In *Proceedings of the 2019 Conference on Empirical Methods in Natural Language Processing (EMNLP)*, pages 3982–3992, 2019.

[SBZ+21] Wahidah Sanusi, Nasiah Badwi, Ahmad Zaki, Sahlan Sidjara, Nurwahidah Sari, Muhammad Isbar Pratama, and Syafruddin Side. Analysis and simulation of SIRS model for dengue fever transmission in South Sulawesi, Indonesia. *Journal of Applied Mathematics*, 2021:1–8, January 2021.

[Wor25] World Health Organization. Dengue and severe dengue fact sheet. Technical report, World Health Organization, August 2025.